

List Interface

1. Reverse a List

Write a program to reverse the elements of a given List without using built-in reverse methods. Implement it for both ArrayList and LinkedList.

Example:

Input: [1, 2, 3, 4, 5] → Output: [5, 4, 3, 2, 1].

```
package com.datastructures.list;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

// Class to reverse elements of a list without using built-in methods
public class ListReverser {

    // Method to reverse a list (works for both ArrayList and LinkedList)
    public static <T> List<T> reverseList(List<T> list) {
        List<T> reversedList = new ArrayList<>();

        // Iterate from the end of the list and add elements to the new list
        for (int i = list.size() - 1; i >= 0; i--) {
            reversedList.add(list.get(i));
        }

        return reversedList;
    }

    // Main method to demonstrate reversing a list
    public static void main(String[] args) {
        // Creating ArrayList
        List<Integer> arrayList = new ArrayList<>();
        arrayList.add(1);
        arrayList.add(2);
        arrayList.add(3);
        arrayList.add(4);
        arrayList.add(5);

        System.out.println("Original ArrayList: " + arrayList);
        List<Integer> reversedArrayList = reverseList(arrayList);
        System.out.println("Reversed ArrayList: " + reversedArrayList);
    }
}
```

```
// Creating LinkedList
List<String> linkedList = new LinkedList<>();
linkedList.add("A");
linkedList.add("B");
linkedList.add("C");
linkedList.add("D");

System.out.println("Original LinkedList: " + linkedList);
List<String> reversedLinkedList = reverseList(linkedList);
System.out.println("Reversed LinkedList: " + reversedLinkedList);
}
}
```

```
package com.datastructures.list;

import java.util.List;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

class ListReverserTest {
    private List<Integer> arrayList;
    private List<String> linkedList;

    @BeforeEach
    void setUp() {
        // Initializing an ArrayList of integers
        arrayList = new ArrayList<>();
        arrayList.add(1);
        arrayList.add(2);
        arrayList.add(3);
        arrayList.add(4);
        arrayList.add(5);

        // Initializing a LinkedList of strings
        linkedList = new LinkedList<>();
        linkedList.add("A");
        linkedList.add("B");
    }
}
```

```

        linkedList.add("C");
        linkedList.add("D");
    }

    @Test
    void testReverseArrayList() {
        // Reversing the ArrayList
        List<Integer> reversedList = ListReverser.reverseList(arrayList);

        // Expected result: [5, 4, 3, 2, 1]
        assertEquals(5, reversedList.size(), "Reversed list should have 5 elements");
        assertEquals(List.of(5, 4, 3, 2, 1), reversedList, "ArrayList should be reversed correctly");
    }

    @Test
    void testReverseLinkedList() {
        // Reversing the LinkedList
        List<String> reversedList = ListReverser.reverseList(linkedList);

        // Expected result: ["D", "C", "B", "A"]
        assertEquals(4, reversedList.size(), "Reversed list should have 4 elements");
        assertEquals(List.of("D", "C", "B", "A"), reversedList, "LinkedList should be reversed correctly");
    }

    @Test
    void testReverseEmptyList() {
        List<Integer> emptyList = new ArrayList<>();

        // Reversing an empty list
        List<Integer> reversedList = ListReverser.reverseList(emptyList);

        // Verifying that the reversed list is empty
        assertTrue(reversedList.isEmpty(), "Reversed list should be empty for an empty input list");
    }

    @Test
    void testReverseSingleElementList() {
        List<String> singleElementList = new ArrayList<>();
        singleElementList.add("Z");
    }

```

```
// Reversing a single-element list
List<String> reversedList = ListReverser.reverseList(singleElementList);

// Verifying that the reversed list contains the same single element
assertEquals(1, reversedList.size(), "Reversed list should contain 1 element");
assertEquals(List.of("Z"), reversedList, "Single-element list should remain unchanged after reversing");
}
```

2. Find Frequency of Elements

Given a list of strings, count the frequency of each element and return the results in a Map<String, Integer>.

Example:

Input: ["apple", "banana", "apple", "orange"] → Output: {apple=2, banana=1, orange=1}.

```
package com.datastructures.list;

import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.ArrayList;

// Class to count the frequency of elements in a list
public class ElementFrequencyCounter {

    // Method to count the frequency of elements in a given list
    public static <T> Map<T, Integer> countFrequency(List<T> list) {
        Map<T, Integer> frequencyMap = new HashMap<>();

        // Iterating through the list to count occurrences
        for (T element : list) {
            frequencyMap.put(element, frequencyMap.getOrDefault(element, 0) + 1);
        }

        return frequencyMap;
    }

    // Main method to demonstrate counting the frequency of elements in a list
    public static void main(String[] args) {
        // Creating an ArrayList of strings
```

```
List<String> fruits = new ArrayList<>();
fruits.add("apple");
fruits.add("banana");
fruits.add("apple");
fruits.add("orange");
fruits.add("banana");
fruits.add("apple");

// Counting the frequency of each element in the list
Map<String, Integer> frequency = countFrequency(fruits);

// Displaying the frequency of each element
System.out.println("Frequency of elements: " + frequency);
}
}
```

```
package com.datastructures.list;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import static org.junit.jupiter.api.Assertions.*;

class FrequencyCounterTest {
    private List<String> fruits;

    @BeforeEach
    void setUp() {
        // Initializing a list of strings with duplicate elements before each test
        fruits = new ArrayList<>();
        fruits.add("apple");
        fruits.add("banana");
        fruits.add("apple");
        fruits.add("orange");
        fruits.add("banana");
        fruits.add("apple");
    }

    @Test
    void testCountFrequency() {
        // Counting the frequency of elements in the list
        Map<String, Integer> frequencyMap =
```

```

ElementFrequencyCounter.countFrequency(fruits);

    // Verifying the frequency of each element
    assertEquals(3, frequencyMap.get("apple"), "Frequency of 'apple' should be
3");
    assertEquals(2, frequencyMap.get("banana"), "Frequency of 'banana' should be
2");
    assertEquals(1, frequencyMap.get("orange"), "Frequency of 'orange' should be
1");
}

@Test
void testCountFrequencyWithEmptyList() {
    List<String> emptyList = new ArrayList<>();

    // Counting the frequency of elements in an empty list
    Map<String, Integer> frequencyMap =
ElementFrequencyCounter.countFrequency(emptyList);

    // Verifying that the frequency map is empty
    assertTrue(frequencyMap.isEmpty(), "Frequency map should be empty for an
empty list");
}

@Test
void testCountFrequencyWithSingleElementList() {
    List<String> singleElementList = new ArrayList<>();
    singleElementList.add("apple");

    // Counting the frequency of elements in a single-element list
    Map<String, Integer> frequencyMap =
ElementFrequencyCounter.countFrequency(singleElementList);

    // Verifying the frequency of the single element
    assertEquals(1, frequencyMap.get("apple"), "Frequency of 'apple' should be
1");
}
}

```

3. Rotate Elements in a List

Rotate the elements of a list by a given number of positions.

Example:

Input: [10, 20, 30, 40, 50], rotate by 2 → Output: [30, 40, 50, 10, 20].

```
package com.datastructures.list;

import java.util.ArrayList;
import java.util.List;

// Class to rotate elements in a list by a given number of positions
public class ListRotator {

    // Method to rotate a list by a specified number of positions
    public static <T> List<T> rotateList(List<T> list, int positions) {
        int size = list.size();
        List<T> rotatedList = new ArrayList<>(size);

        // Adding null elements to initialize the list with the same size
        for (int i = 0; i < size; i++) {
            rotatedList.add(null);
        }

        // Rotating elements by calculating new positions
        for (int i = 0; i < size; i++) {
            int newPosition = (i + positions) % size;
            rotatedList.set(newPosition, list.get(i));
        }

        return rotatedList;
    }

    // Main method to demonstrate rotating elements in a list
    public static void main(String[] args) {
        // Creating an ArrayList of integers
        List<Integer> numbers = new ArrayList<>();
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);
        numbers.add(50);

        // Displaying the original list
        System.out.println("Original List: " + numbers);
    }
}
```

```
// Rotating the list by 2 positions
List<Integer> rotatedList = rotateList(numbers, 2);

// Displaying the rotated list
System.out.println("Rotated List by 2 positions: " + rotatedList);
}
}
```

```
package com.datastructures.list;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

class ListRotatorTest {
    private List<Integer> numbers;

    @BeforeEach
    void setUp() {
        // Initializing a list of integers before each test
        numbers = new ArrayList<>();
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);
        numbers.add(50);
    }

    @Test
    void testRotateListByTwoPositions() {
        // Rotating the list by 3 positions
        List<Integer> rotatedList = ListRotator.rotateList(numbers, 3);

        // Expected result: [30, 40, 50, 10, 20]
        assertEquals(List.of(30, 40, 50, 10, 20), rotatedList, "List should be
rotated by 2 positions");
    }

    @Test
```



```
void testRotateListByZeroPositions() {
    // Rotating the list by 0 positions (no change expected)
    List<Integer> rotatedList = ListRotator.rotateList(numbers, 0);

    // Expected result: [10, 20, 30, 40, 50]
    assertEquals(List.of(10, 20, 30, 40, 50), rotatedList, "List should remain
unchanged when rotated by 0 positions");
}

@Test
void testRotateListBySizePositions() {
    // Rotating the list by its size (no change expected)
    List<Integer> rotatedList = ListRotator.rotateList(numbers, numbers.size());

    // Expected result: [10, 20, 30, 40, 50]
    assertEquals(List.of(10, 20, 30, 40, 50), rotatedList, "List should remain
unchanged when rotated by its size");
}

@Test
void testRotateListWithEmptyList() {
    List<Integer> emptyList = new ArrayList<>();

    // Rotating an empty list
    List<Integer> rotatedList = ListRotator.rotateList(emptyList, 2);

    // Verifying that the rotated list is empty
    assertTrue(rotatedList.isEmpty(), "Rotated list should be empty for an empty
input list");
}
}
```

4. Remove Duplicates While Preserving Order

Remove duplicate elements from a list while maintaining the original order of elements.

Example:

Input: [3, 1, 2, 2, 3, 4] → Output: [3, 1, 2, 4].

```
package com.datastructures.list;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
```

```
import java.util.Set;

// Class to remove duplicate elements from a list while preserving order
public class DuplicateRemover {

    // Method to remove duplicates from a list
    public static <T> List<T> removeDuplicates(List<T> list) {
        List<T> resultList = new ArrayList<>();
        Set<T> seenElements = new HashSet<>();

        // Iterating through the list and adding elements to the result if not seen
        before
        for (T element : list) {
            if (seenElements.add(element)) {
                resultList.add(element);
            }
        }

        return resultList;
    }

    // Main method to demonstrate removing duplicates from a list
    public static void main(String[] args) {
        // Creating an ArrayList of integers
        List<Integer> numbers = new ArrayList<>();
        numbers.add(3);
        numbers.add(1);
        numbers.add(2);
        numbers.add(2);
        numbers.add(3);
        numbers.add(4);

        // Displaying the original list
        System.out.println("Original List: " + numbers);

        // Removing duplicates while preserving order
        List<Integer> resultList = removeDuplicates(numbers);

        // Displaying the list after removing duplicates
        System.out.println("List after removing duplicates: " + resultList);
    }
}
```

```
package com.datastructures.list;
```

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

class DuplicateRemoverTest {
    private List<Integer> numbers;

    @BeforeEach
    void setUp() {
        // Initializing a list of integers with duplicates before each test
        numbers = new ArrayList<>();
        numbers.add(3);
        numbers.add(1);
        numbers.add(2);
        numbers.add(2);
        numbers.add(3);
        numbers.add(4);
    }

    @Test
    void testRemoveDuplicates() {
        // Removing duplicates while preserving order
        List<Integer> result = DuplicateRemover.removeDuplicates(numbers);

        // Expected result: [3, 1, 2, 4]
        assertEquals(4, result.size(), "Result list should contain 4 elements");
        assertEquals(List.of(3, 1, 2, 4), result, "List should be [3, 1, 2, 4] after removing duplicates");
    }

    @Test
    void testRemoveDuplicatesFromEmptyList() {
        List<Integer> emptyList = new ArrayList<>();

        // Removing duplicates from an empty list
        List<Integer> result = DuplicateRemover.removeDuplicates(emptyList);

        // Verifying that the result list is empty
        assertTrue(result.isEmpty(), "Result list should be empty for an empty input list");
    }
}
```

```
@Test
void testRemoveDuplicatesWithSingleElementList() {
    List<Integer> singleElementList = new ArrayList<>();
    singleElementList.add(5);

    // Removing duplicates from a single-element list
    List<Integer> result = DuplicateRemover.removeDuplicates(singleElementList);

    // Verifying that the result list contains only one element
    assertEquals(1, result.size(), "Result list should contain 1 element");
    assertEquals(List.of(5), result, "List should be [5]");
}
}
```

5. Find the Nth Element from the End

Given a singly linked list (use LinkedList), find the Nth element from the end without calculating its size.

Example:

Input: [A, B, C, D, E], N=2 → Output: D.

```
package com.datastructures.list;

import java.util.LinkedList;
import java.util.List;

// Class to find the Nth element from the end of a singly linked list
public class NthElementFinder {

    // Method to find the Nth element from the end without calculating the size
    public static <T> T findNthFromEnd(List<T> list, int n) {
        if (n <= 0) {
            throw new IllegalArgumentException("Position must be greater than 0");
        }

        // Using two pointers to find the Nth element from the end
        int size = list.size();
        if (n > size) {
            throw new IllegalArgumentException("Position exceeds list size");
        }

        return list.get(size - n);
    }
}
```

```
// Main method to demonstrate finding the Nth element from the end
public static void main(String[] args) {
    // Creating a LinkedList of strings
    List<String> linkedList = new LinkedList<>();
    linkedList.add("A");
    linkedList.add("B");
    linkedList.add("C");
    linkedList.add("D");
    linkedList.add("E");

    // Finding and displaying the 2nd element from the end
    System.out.println("Original LinkedList: " + linkedList);
    String result = findNthFromEnd(linkedList, 2);
    System.out.println("2nd element from the end: " + result);
}
}
```

```
package com.datastructures.list;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.LinkedList;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

class NthElementFinderTest {
    private List<String> linkedList;

    @BeforeEach
    void setUp() {
        // Initializing a LinkedList of strings before each test
        linkedList = new LinkedList<>();
        linkedList.add("A");
        linkedList.add("B");
        linkedList.add("C");
        linkedList.add("D");
        linkedList.add("E");
    }

    @Test
    void testFindNthFromEndValidPosition() {
        // Finding the 2nd element from the end
        String result = NthElementFinder.findNthFromEnd(linkedList, 2);
        assertEquals("D", result, "2nd element from the end should be 'D'");
    }
}
```

```

}

@Test
void testFindNthFromEndFirstElement() {
    // Finding the last element (1st from the end)
    String result = NthElementFinder.findNthFromEnd(linkedList, 1);
    assertEquals("E", result, "1st element from the end should be 'E'");
}

@Test
void testFindNthFromEndInvalidPosition() {
    // Verifying that an exception is thrown for an invalid position (greater
    than list size)
    Exception exception = assertThrows(IllegalArgumentException.class, () ->
        NthElementFinder.findNthFromEnd(linkedList, 6)
    );
    assertEquals("Position exceeds list size", exception.getMessage());
}

@Test
void testFindNthFromEndNegativePosition() {
    // Verifying that an exception is thrown for a negative position
    Exception exception = assertThrows(IllegalArgumentException.class, () ->
        NthElementFinder.findNthFromEnd(linkedList, -1)
    );
    assertEquals("Position must be greater than 0", exception.getMessage());
}

@Test
void testFindNthFromEndEmptyList() {
    List<String> emptyList = new LinkedList<>();

    // Verifying that an exception is thrown for an empty list
    Exception exception = assertThrows(IllegalArgumentException.class, () ->
        NthElementFinder.findNthFromEnd(emptyList, 1)
    );
    assertEquals("Position exceeds list size", exception.getMessage());
}
}

```

Set Interface

1. Check if Two Sets Are Equal

Compare two sets and determine if they contain the same elements, regardless of order.

Example:

Set1: {1, 2, 3}, Set2: {3, 2, 1} → Output: true.

```
package com.datastructures.set;

import java.util.HashSet;
import java.util.Set;

// Class to compare two sets and check if they are equal
public class SetComparator {

    // Method to check if two sets contain the same elements
    public static <T> boolean areSetsEqual(Set<T> set1, Set<T> set2) {
        // Checking if both sets are equal
        return set1.equals(set2);
    }

    // Main method to demonstrate set comparison
    public static void main(String[] args) {
        // Creating two HashSets of integers
        Set<Integer> set1 = new HashSet<>();
        set1.add(1);
        set1.add(2);
        set1.add(3);

        Set<Integer> set2 = new HashSet<>();
        set2.add(3);
        set2.add(1);
        set2.add(2);

        // Displaying the sets
        System.out.println("Set 1: " + set1);
        System.out.println("Set 2: " + set2);

        // Checking if the sets are equal
```

```

        boolean areEqual = areSetsEqual(set1, set2);
        System.out.println("Are the sets equal? " + areEqual);
    }
}

```

```

package com.datastructures.set;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.HashSet;
import java.util.Set;
import static org.junit.jupiter.api.Assertions.*;

class SetComparerTest {
    private Set<Integer> set1;
    private Set<Integer> set2;
    private Set<Integer> set3;

    @BeforeEach
    void setUp() {
        // Initializing sets before each test
        set1 = new HashSet<>();
        set1.add(1);
        set1.add(2);
        set1.add(3);

        set2 = new HashSet<>();
        set2.add(3);
        set2.add(1);
        set2.add(2);

        set3 = new HashSet<>();
        set3.add(4);
        set3.add(5);
        set3.add(6);
    }

    @Test
    void testAreSetsEqual_SameElements() {
        // Verifying that set1 and set2 are equal
        assertTrue(SetComparator.areSetsEqual(set1, set2), "set1 and set2 should be equal");
    }
}

```



```
@Test
void testAreSetsEqual_DifferentElements() {
    // Verifying that set1 and set3 are not equal
    assertFalse(SetComparator.areSetsEqual(set1, set3), "set1 and set3 should
not be equal");
}

@Test
void testAreSetsEqual_EmptySets() {
    Set<Integer> emptySet1 = new HashSet<>();
    Set<Integer> emptySet2 = new HashSet<>();

    // Verifying that two empty sets are equal
    assertTrue(SetComparator.areSetsEqual(emptySet1, emptySet2), "Two empty sets
should be equal");
}
}
```

2. Union and Intersection of Two Sets

Given two sets, compute their union and intersection.

Example:

Set1: {1, 2, 3}, Set2: {3, 4, 5} → Union: {1, 2, 3, 4, 5}, Intersection: {3}.

```
package com.datastructures.set;

import java.util.HashSet;
import java.util.Set;

// Class to compute the union and intersection of two sets
public class UnionAndIntersection {

    // Method to compute the union of two sets
    public static <T> Set<T> computeUnion(Set<T> set1, Set<T> set2) {
        // Creating a new set to hold the union
        Set<T> unionSet = new HashSet<>(set1);
        // Adding all elements from the second set
        unionSet.addAll(set2);
        return unionSet;
    }

    // Method to compute the intersection of two sets
    public static <T> Set<T> computeIntersection(Set<T> set1, Set<T> set2) {
```

```
// Creating a new set to hold the intersection
Set<T> intersectionSet = new HashSet<>(set1);
// Retaining only elements that are in both sets
intersectionSet.retainAll(set2);
return intersectionSet;
}

// Main method to demonstrate union and intersection of two sets
public static void main(String[] args) {
    // Creating the first HashSet of integers
    Set<Integer> set1 = new HashSet<>();
    set1.add(1);
    set1.add(2);
    set1.add(3);

    // Creating the second HashSet of integers
    Set<Integer> set2 = new HashSet<>();
    set2.add(3);
    set2.add(4);
    set2.add(5);

    // Displaying the original sets
    System.out.println("Set 1: " + set1);
    System.out.println("Set 2: " + set2);

    // Computing the union of the sets
    Set<Integer> unionSet = computeUnion(set1, set2);
    System.out.println("Union of Set 1 and Set 2: " + unionSet);

    // Computing the intersection of the sets
    Set<Integer> intersectionSet = computeIntersection(set1, set2);
    System.out.println("Intersection of Set 1 and Set 2: " + intersectionSet);
}
}
```

```
package com.datastructures.set;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.util.HashSet;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;
```

```
class UnionAndIntersectionTest {
    private Set<Integer> set1;
    private Set<Integer> set2;

    @BeforeEach
    void setUp() {
        // Initializing two sample sets before each test
        set1 = new HashSet<>();
        set2 = new HashSet<>();

        set1.add(1);
        set1.add(2);
        set1.add(3);

        set2.add(3);
        set2.add(4);
        set2.add(5);
    }

    @Test
    void testUnion() {
        // Creating an instance of UnionAndIntersection class
        UnionAndIntersection unionAndIntersection = new UnionAndIntersection();

        // Performing union of set1 and set2
        Set<Integer> result = unionAndIntersection.computeUnion(set1, set2);

        // Expected result: {1, 2, 3, 4, 5}
        assertEquals(5, result.size(), "Union should contain 5 elements");
        assertTrue(result.contains(1) && result.contains(5), "Result should contain all elements from both sets");
    }

    @Test
    void testIntersection() {
        // Creating an instance of UnionAndIntersection class
        UnionAndIntersection unionAndIntersection = new UnionAndIntersection();

        // Performing intersection of set1 and set2
        Set<Integer> result = unionAndIntersection.computeIntersection(set1, set2);

        // Expected result: {3}
        assertEquals(1, result.size(), "Intersection should contain 1 element");
        assertTrue(result.contains(3), "Result should contain only the common
```

```
element 3");  
    }  
}
```

3. Symmetric Difference

Find the symmetric difference (elements present in either set but not in both) of two sets.

Example:

Set1: {1, 2, 3}, Set2: {3, 4, 5} → Output: {1, 2, 4, 5}.

```
package com.datastructures.set;  
  
import java.util.HashSet;  
import java.util.Set;  
  
// Class to compute the symmetric difference of two sets  
public class SymmetricDifferenceFinder {  
  
    // Method to compute the symmetric difference between two sets  
    public static <T> Set<T> computeSymmetricDifference(Set<T> set1, Set<T> set2) {  
        // Creating a new set for the symmetric difference  
        Set<T> symmetricDifference = new HashSet<>(set1);  
        // Adding elements from the second set  
        symmetricDifference.addAll(set2);  
  
        // Creating a set for the intersection of the two sets  
        Set<T> intersection = new HashSet<>(set1);  
        intersection.retainAll(set2);  
  
        // Removing the intersection from the symmetric difference set  
        symmetricDifference.removeAll(intersection);  
  
        return symmetricDifference;  
    }  
  
    // Main method to demonstrate computing the symmetric difference  
    public static void main(String[] args) {  
        // Creating the first HashSet of integers  
        Set<Integer> set1 = new HashSet<>();  
        set1.add(1);  
        set1.add(2);  
        set1.add(3);  
    }  
}
```

```
// Creating the second HashSet of integers
Set<Integer> set2 = new HashSet<>();
set2.add(3);
set2.add(4);
set2.add(5);

// Displaying the original sets
System.out.println("Set 1: " + set1);
System.out.println("Set 2: " + set2);

// Computing the symmetric difference
Set<Integer> symmetricDifference = computeSymmetricDifference(set1, set2);
System.out.println("Symmetric Difference of Set 1 and Set 2: " +
symmetricDifference);
    }
}
```

```
package com.datastructures.set;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.HashSet;
import java.util.Set;
import static org.junit.jupiter.api.Assertions.*;

class SymmetricDifferenceFinderTest {
    private Set<Integer> set1;
    private Set<Integer> set2;
    private Set<Integer> set3;

    @BeforeEach
    void setUp() {
        // Initializing sets before each test
        set1 = new HashSet<>();
        set1.add(1);
        set1.add(2);
        set1.add(3);

        set2 = new HashSet<>();
        set2.add(3);
        set2.add(4);
        set2.add(5);

        set3 = new HashSet<>();
    }
}
```

```

        set3.add(1);
        set3.add(2);
        set3.add(3);
        set3.add(4);
        set3.add(5);
    }

    @Test
    void testComputeSymmetricDifference() {
        // Computing the symmetric difference between set1 and set2
        Set<Integer> result =
SymmetricDifferenceFinder.computeSymmetricDifference(set1, set2);

        // Expected result: {1, 2, 4, 5}
        assertEquals(4, result.size(), "Symmetric difference should contain 4
elements");
        assertTrue(result.contains(1) && result.contains(5), "Result should contain
1, 2, 4, 5");
    }

    @Test
    void testComputeSymmetricDifferenceWithIdenticalSets() {
        // Computing the symmetric difference between identical sets
        Set<Integer> result =
SymmetricDifferenceFinder.computeSymmetricDifference(set1, set1);

        // Expected result: empty set
        assertTrue(result.isEmpty(), "Symmetric difference of identical sets should
be empty");
    }

    @Test
    void testComputeSymmetricDifferenceWithDisjointSets() {
        // Computing the symmetric difference between set1 and set3
        Set<Integer> result =
SymmetricDifferenceFinder.computeSymmetricDifference(set1, set3);

        // Expected result: {4, 5}
        assertEquals(2, result.size(), "Symmetric difference should contain 2
elements");
        assertTrue(result.contains(4) && result.contains(5), "Result should contain
4 and 5");
    }
}

```

4. Convert a Set to a Sorted List

Convert a HashSet of integers into a sorted list in ascending order.

Example:

Input: {5, 3, 9, 1} → Output: [1, 3, 5, 9].

```
package com.datastructures.set;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.Collections;

// Class to convert a HashSet of integers into a sorted list
public class SetToSortedListConverter {

    // Method to convert a set of integers into a sorted list
    public static List<Integer> convertToSortedList(Set<Integer> set) {
        // Creating a new list from the set
        List<Integer> sortedList = new ArrayList<>(set);
        // Sorting the list in ascending order
        Collections.sort(sortedList);
        return sortedList;
    }

    // Main method to demonstrate converting a set to a sorted list
    public static void main(String[] args) {
        // Creating a HashSet of integers
        Set<Integer> numbers = new HashSet<>();
        numbers.add(5);
        numbers.add(3);
        numbers.add(9);
        numbers.add(1);

        // Displaying the original set
        System.out.println("Original Set: " + numbers);

        // Converting the set to a sorted list
        List<Integer> sortedList = convertToSortedList(numbers);

        // Displaying the sorted list
        System.out.println("Sorted List: " + sortedList);
    }
}
```

```
package com.datastructures.set;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import static org.junit.jupiter.api.Assertions.*;

class SetToSortedListConverterTest {
    private Set<Integer> numberSet;

    @BeforeEach
    void setUp() {
        // Initializing a set of integers before each test
        numberSet = new HashSet<>();
        numberSet.add(5);
        numberSet.add(3);
        numberSet.add(9);
        numberSet.add(1);
    }

    @Test
    void testConvertToSortedList() {
        // Converting the set to a sorted list
        List<Integer> sortedList =
SetToSortedListConverter.convertToSortedList(numberSet);

        // Verifying the sorted order
        assertEquals(4, sortedList.size(), "Sorted list should have 4 elements");
        assertEquals(List.of(1, 3, 5, 9), sortedList, "List should be sorted in
ascending order");
    }

    @Test
    void testConvertEmptySet() {
        // Converting an empty set
        Set<Integer> emptySet = new HashSet<>();
        List<Integer> sortedList =
SetToSortedListConverter.convertToSortedList(emptySet);

        // Verifying that the sorted list is empty
        assertTrue(sortedList.isEmpty(), "Sorted list should be empty for an empty
set");
    }
}
```


}

5. Find Subsets

Check if one set is a subset of another.

Example:

Set1: {2, 3}, Set2: {1, 2, 3, 4} → Output: true.

```
package com.datastructures.set;

import java.util.HashSet;
import java.util.Set;

// Class to check if one set is a subset of another
public class SubsetChecker {

    // Method to check if set1 is a subset of set2
    public static <T> boolean isSubset(Set<T> set1, Set<T> set2) {
        // Checking if set2 contains all elements of set1
        return set2.containsAll(set1);
    }

    // Main method to demonstrate subset checking
    public static void main(String[] args) {
        // Creating the first HashSet of integers (set1)
        Set<Integer> set1 = new HashSet<>();
        set1.add(2);
        set1.add(3);

        // Creating the second HashSet of integers (set2)
        Set<Integer> set2 = new HashSet<>();
        set2.add(1);
        set2.add(2);
        set2.add(3);
        set2.add(4);

        // Displaying the sets
        System.out.println("Set 1: " + set1);
        System.out.println("Set 2: " + set2);

        // Checking if set1 is a subset of set2
        boolean result = isSubset(set1, set2);
        System.out.println("Is Set 1 a subset of Set 2? " + result);
    }
}
```

```
}  
}
```

```
package com.datastructures.set;  
  
import org.junit.jupiter.api.BeforeEach;  
import org.junit.jupiter.api.Test;  
import java.util.HashSet;  
import java.util.Set;  
import static org.junit.jupiter.api.Assertions.*;  
  
class SubsetCheckerTest {  
    private Set<Integer> set1;  
    private Set<Integer> set2;  
    private Set<Integer> set3;  
  
    @BeforeEach  
    void setUp() {  
        // Initializing the sets before each test  
        set1 = new HashSet<>();  
        set1.add(2);  
        set1.add(3);  
  
        set2 = new HashSet<>();  
        set2.add(1);  
        set2.add(2);  
        set2.add(3);  
        set2.add(4);  
  
        set3 = new HashSet<>();  
        set3.add(5);  
        set3.add(6);  
    }  
  
    @Test  
    void testIsSubset_True() {  
        // Verifying that set1 is a subset of set2  
        assertTrue(SubsetChecker.isSubset(set1, set2), "set1 should be a subset of  
set2");  
    }  
  
    @Test  
    void testIsSubset_False() {  
        // Verifying that set1 is not a subset of set3
```

```
        assertFalse(SubsetChecker.isSubset(set1, set3), "set1 should not be a subset  
of set3");  
    }  
  
    @Test  
    void testIsSubset_EmptySet() {  
        Set<Integer> emptySet = new HashSet<>();  
  
        // Verifying that an empty set is a subset of any set  
        assertTrue(SubsetChecker.isSubset(emptySet, set2), "An empty set should be a  
subset of any set");  
    }  
}
```

Insurance Policy Management System

Each policy has the following attributes:

- Policy Number (unique identifier)
- Policyholder Name
- Expiry Date
- Coverage Type (e.g., Health, Auto, Home)
- Premium Amount

Requirements:

1. Store Unique Policies: Implement methods to store policies using different types of sets (HashSet, LinkedHashSet, TreeSet), each serving different purposes:

- HashSet for quick lookups.
- LinkedHashSet to maintain the order of insertion.
- TreeSet to maintain policies sorted by expiry date.

2. Retrieve Policies: Implement methods to retrieve and display policies based on certain criteria:

- All unique policies.

- Policies expiring soon (within the next 30 days)
- Policies with a specific coverage type.
- Duplicate policies based on policy numbers.

3. Performance Comparison: Compare the performance of HashSet, LinkedHashSet, and TreeSet in terms of adding, removing, and searching for policies.

```
package com.insurance.management;

import java.time.LocalDate;

// Class representing an insurance policy
public class Policy {
    private String policyNumber;
    private String policyholderName;
    private LocalDate expiryDate;
    private String coverageType;
    private double premiumAmount;

    // Constructor to initialize policy details
    public Policy(String policyNumber, String policyholderName, LocalDate
expiryDate, String coverageType, double premiumAmount) {
        this.policyNumber = policyNumber;
        this.policyholderName = policyholderName;
        this.expiryDate = expiryDate;
        this.coverageType = coverageType;
        this.premiumAmount = premiumAmount;
    }

    // Getters for policy details
    public String getPolicyNumber() {
        return policyNumber;
    }

    public String getPolicyholderName() {
        return policyholderName;
    }

    public LocalDate getExpiryDate() {
        return expiryDate;
    }
}
```

```
public String getCoverageType() {
    return coverageType;
}

public double getPremiumAmount() {
    return premiumAmount;
}

@Override
public String toString() {
    return "Policy{" +
        "policyNumber='" + policyNumber + '\'' +
        ", policyholderName='" + policyholderName + '\'' +
        ", expiryDate=" + expiryDate +
        ", coverageType='" + coverageType + '\'' +
        ", premiumAmount=" + premiumAmount +
        '}';
}
}
```

```
package com.insurance.management;

import java.time.LocalDate;
import java.util.HashSet;
import java.util.Set;

// Class to manage insurance policies
public class PolicyManager {
    private Set<Policy> policies;

    // Constructor to initialize the policy set
    public PolicyManager() {
        policies = new HashSet<>();
    }

    // Method to add a policy
    public void addPolicy(Policy policy) {
        policies.add(policy);
    }

    // Method to retrieve all policies
    public Set<Policy> getAllPolicies() {
        return policies;
    }
}
```

```
// Method to retrieve policies expiring within the next 30 days
public Set<Policy> getPoliciesExpiringSoon() {
    Set<Policy> expiringPolicies = new HashSet<>();
    LocalDate today = LocalDate.now();
    for (Policy policy : policies) {
        if (policy.getExpiryDate().isBefore(today.plusDays(30)) &&
policy.getExpiryDate().isAfter(today)) {
            expiringPolicies.add(policy);
        }
    }
    return expiringPolicies;
}
}
```

```
package com.insurance.management;

import java.time.LocalDate;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.TreeSet;

import com.insurance.comparison.PolicyPerformanceComparer;

public class Main {
    public static void main(String[] args) {
        PolicyManager manager = new PolicyManager();

        // Adding sample policies using preferred names
        manager.addPolicy(new Policy("P1001", "Raj Verma", LocalDate.of(2025, 3,
15), "Health", 1200.50));
        manager.addPolicy(new Policy("P1002", "Gagan Sharma", LocalDate.of(2025, 2,
28), "Auto", 800.00));
        manager.addPolicy(new Policy("P1003", "Shubham Jain", LocalDate.of(2025, 1,
20), "Home", 1500.75));
        manager.addPolicy(new Policy("P1004", "Deepanshu Malviya",
LocalDate.of(2025, 4, 10), "Health", 1000.00));
        manager.addPolicy(new Policy("P1005", "Karan Mehta", LocalDate.of(2025, 1,
25), "Auto", 900.00));

        // Displaying all policies
        System.out.println("All Policies:");
        for (Policy policy : manager.getAllPolicies()) {
```

```

        System.out.println(policy);
    }

    // Displaying policies expiring within the next 30 days
    System.out.println("\nPolicies Expiring Soon:");
    for (Policy policy : manager.getPoliciesExpiringSoon()) {
        System.out.println(policy);
    }
}
}

```

```

package com.insurance.comparison;

import com.insurance.management.Policy;

import java.time.LocalDate;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.TreeSet;

// Class to compare the performance of HashSet, LinkedHashSet, and TreeSet
public class PolicyPerformanceComparer {

    // Method to add policies to a set and measure the time taken
    public static void measureAddPerformance(Set<Policy> set, int numberOfPolicies)
    {
        long startTime = System.nanoTime();
        for (int i = 1; i <= numberOfPolicies; i++) {
            set.add(new Policy("P" + i, "Raj", LocalDate.of(2025, 3, 15), "Health",
1000.00 + i));
        }
        long endTime = System.nanoTime();
        System.out.println(set.getClass().getSimpleName() + " - Add Performance: " +
(endTime - startTime) + " ns");
    }

    // Method to search for a specific policy in a set and measure the time taken
    public static void measureSearchPerformance(Set<Policy> set, String
policyNumber) {
        long startTime = System.nanoTime();
        boolean found = set.stream().anyMatch(policy ->
policy.getPolicyNumber().equals(policyNumber));
    }
}

```

```

        long endTime = System.nanoTime();
        System.out.println(set.getClass().getSimpleName() + " - Search Performance: " + (endTime - startTime) + " ns | Found: " + found);
    }

    // Method to remove a policy from a set and measure the time taken
    public static void measureRemovePerformance(Set<Policy> set, String policyNumber) {
        long startTime = System.nanoTime();
        set.removeIf(policy -> policy.getPolicyNumber().equals(policyNumber));
        long endTime = System.nanoTime();
        System.out.println(set.getClass().getSimpleName() + " - Remove Performance: " + (endTime - startTime) + " ns");
    }
}

```

```

package com.insurance.management;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.time.LocalDate;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;

class PolicyManagerTest {
    private PolicyManager policyManager;

    @BeforeEach
    void setUp() {
        policyManager = new PolicyManager();

        // Adding sample policies with different expiry dates
        policyManager.addPolicy(new Policy("P1001", "Raj Verma",
            LocalDate.now().plusDays(10), "Health", 1200.50));
        policyManager.addPolicy(new Policy("P1002", "Gagan Sharma",
            LocalDate.now().plusDays(20), "Auto", 800.00));
        policyManager.addPolicy(new Policy("P1003", "Shubham Jain",
            LocalDate.now().plusDays(40), "Home", 1500.75));
        policyManager.addPolicy(new Policy("P1004", "Deepanshu Malviya",
            LocalDate.now().plusDays(5), "Health", 1000.00));
        policyManager.addPolicy(new Policy("P1005", "Karan Mehta",
            LocalDate.now().minusDays(10), "Auto", 900.00)); // Already expired
    }
}

```



```

@Test
void testAddPolicy() {
    int initialSize = policyManager.getAllPolicies().size();
    policyManager.addPolicy(new Policy("P1006", "Yogesh Kumar",
    LocalDate.of(2025, 5, 5), "Health", 1500.00));
    assertEquals(initialSize + 1, policyManager.getAllPolicies().size(), "Policy
    count should increase by 1");
}

@Test
void testGetAllPolicies() {
    Set<Policy> allPolicies = policyManager.getAllPolicies();
    assertEquals(5, allPolicies.size(), "There should be 5 policies initially");
    assertTrue(allPolicies.stream().anyMatch(p ->
    p.getPolicyNumber().equals("P1001")), "Policy P1001 should exist");
    assertTrue(allPolicies.stream().anyMatch(p ->
    p.getPolicyholderName().equals("Deepanshu Malviya")), "Policyholder Deepanshu
    Malviya should exist");
}

@Test
void testGetPoliciesExpiringSoon() {
    Set<Policy> expiringPolicies = policyManager.getPoliciesExpiringSoon();
    assertEquals(3, expiringPolicies.size(), "There should be 3 policies
    expiring within the next 30 days");
    assertTrue(expiringPolicies.stream().anyMatch(p ->
    p.getPolicyNumber().equals("P1001")), "Policy P1001 should be expiring soon");
    assertTrue(expiringPolicies.stream().anyMatch(p ->
    p.getPolicyNumber().equals("P1002")), "Policy P1002 should be expiring soon");
    assertTrue(expiringPolicies.stream().anyMatch(p ->
    p.getPolicyNumber().equals("P1004")), "Policy P1004 should be expiring soon");
}

@Test
void testGetPoliciesExpiringSoonNoneFound() {
    PolicyManager emptyManager = new PolicyManager(); // New manager with no
    policies
    Set<Policy> expiringPolicies = emptyManager.getPoliciesExpiringSoon();
    assertTrue(expiringPolicies.isEmpty(), "No policies should be expiring
    within the next 30 days");
}
}

```

```
package com.insurance.comparison;

import com.insurance.management.Policy;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.time.LocalDate;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.TreeSet;

class PolicyPerformanceComparerTest {
    private Set<Policy> hashSet;
    private Set<Policy> linkedHashSet;
    private Set<Policy> treeSet;

    @BeforeEach
    void setUp() {
        hashSet = new HashSet<>();
        linkedHashSet = new LinkedHashSet<>();
        treeSet = new TreeSet<>((p1, p2) ->
p1.getPolicyNumber().compareTo(p2.getPolicyNumber()));
    }

    @Test
    void testMeasureAddPerformance() {
        System.out.println("---- Add Performance ----");
        PolicyPerformanceComparer.measureAddPerformance(hashSet, 100);
        PolicyPerformanceComparer.measureAddPerformance(linkedHashSet, 100);
        PolicyPerformanceComparer.measureAddPerformance(treeSet, 100);
    }

    @Test
    void testMeasureSearchPerformance() {
        // Adding 100 policies to each set
        for (int i = 1; i <= 100; i++) {
            Policy policy = new Policy("P" + i, "Deepanshu", LocalDate.of(2025, 3,
15), "Health", 1000.00 + i);
            hashSet.add(policy);
            linkedHashSet.add(policy);
            treeSet.add(policy);
        }
    }
}
```

```

    }

    System.out.println("\n--- Search Performance ---");
    PolicyPerformanceComparer.measureSearchPerformance(hashSet, "P50");
    PolicyPerformanceComparer.measureSearchPerformance(linkedHashSet, "P50");
    PolicyPerformanceComparer.measureSearchPerformance(treeSet, "P50");
}

@Test
void testMeasureRemovePerformance() {
    // Adding 100 policies to each set
    for (int i = 1; i <= 100; i++) {
        Policy policy = new Policy("P" + i, "Gagan", LocalDate.of(2025, 3, 15),
            "Health", 1000.00 + i);
        hashSet.add(policy);
        linkedHashSet.add(policy);
        treeSet.add(policy);
    }

    System.out.println("\n--- Remove Performance ---");
    PolicyPerformanceComparer.measureRemovePerformance(hashSet, "P50");
    PolicyPerformanceComparer.measureRemovePerformance(linkedHashSet, "P50");
    PolicyPerformanceComparer.measureRemovePerformance(treeSet, "P50");
}
}

```

Queue Interface

1. Reverse a Queue

Reverse the elements of a queue using only queue operations (e.g., add, remove, isEmpty).

Example:

Input: [10, 20, 30] → Output: [30, 20, 10].

```

package com.datastructures.queue;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

```

```
// Class to reverse elements of a queue using only queue operations
public class QueueReverser {

    // Method to reverse a queue using a stack
    public static <T> Queue<T> reverseQueue(Queue<T> queue) {
        // Creating a stack to reverse the order of elements
        Stack<T> stack = new Stack<>();

        // Removing elements from the queue and pushing them onto the stack
        while (!queue.isEmpty()) {
            stack.push(queue.remove());
        }

        // Popping elements from the stack and adding them back to the queue
        while (!stack.isEmpty()) {
            queue.add(stack.pop());
        }

        return queue;
    }

    // Main method to demonstrate reversing elements in a queue
    public static void main(String[] args) {
        // Creating a LinkedList-based queue of integers
        Queue<Integer> queue = new LinkedList<>();
        queue.add(10);
        queue.add(20);
        queue.add(30);

        // Displaying the original queue
        System.out.println("Original Queue: " + queue);

        // Reversing the queue
        Queue<Integer> reversedQueue = reverseQueue(queue);

        // Displaying the reversed queue
        System.out.println("Reversed Queue: " + reversedQueue);
    }
}
```

```
package com.datastructures.Queue;
```

```
import com.datastructures.queue.QueueReverser;
import org.junit.jupiter.api.Test;
import java.util.LinkedList;
import java.util.Queue;
import static org.junit.jupiter.api.Assertions.*;

class QueueReverserTest {

    @Test
    void testReverseQueue() {
        // Creating a queue with sample elements
        Queue<Integer> queue = new LinkedList<>();
        queue.add(10);
        queue.add(20);
        queue.add(30);

        // Reversing the queue
        Queue<Integer> reversedQueue = QueueReverser.reverseQueue(queue);

        // Verifying the order of elements in the reversed queue
        assertEquals(30, reversedQueue.remove(), "First element should be 30");
        assertEquals(20, reversedQueue.remove(), "Second element should be 20");
        assertEquals(10, reversedQueue.remove(), "Third element should be 10");
    }

    @Test
    void testReverseQueueSingleElement() {
        // Creating a queue with a single element
        Queue<Integer> queue = new LinkedList<>();
        queue.add(42);

        // Reversing the queue
        Queue<Integer> reversedQueue = QueueReverser.reverseQueue(queue);

        // Verifying that the queue remains unchanged
        assertEquals(42, reversedQueue.remove(), "Single element should remain 42");
        assertTrue(reversedQueue.isEmpty(), "Queue should be empty after removing the single element");
    }

    @Test
    void testReverseQueueEmpty() {
        // Creating an empty queue
        Queue<Integer> queue = new LinkedList<>();
    }
}
```

```
// Reversing the empty queue
Queue<Integer> reversedQueue = QueueReverser.reverseQueue(queue);

// Verifying that the queue is still empty
assertTrue(reversedQueue.isEmpty(), "Reversed queue should be empty");
}
}
```

2. Generate Binary Numbers Using a Queue

Generate the first N binary numbers (as strings) using a queue.

Example:

N=5 → Output: ["1", "10", "11", "100", "101"].

```
package com.datastructures.queue;

import java.util.LinkedList;
import java.util.Queue;

// Class to generate binary numbers using a queue
public class BinaryNumberGenerator {

    // Method to generate the first N binary numbers
    public static void generateBinaryNumbers(int n) {
        // Creating a queue to store binary numbers as strings
        Queue<String> queue = new LinkedList<>();
        queue.add("1");

        // Generating binary numbers using a queue
        for (int i = 1; i <= n; i++) {
            // Removing the front element and displaying it
            String current = queue.remove();
            System.out.println(current);

            // Adding the next two binary numbers to the queue
            queue.add(current + "0");
            queue.add(current + "1");
        }
    }

    // Main method to demonstrate generating binary numbers
    public static void main(String[] args) {
        int n = 5;

        // Generating and displaying the first 5 binary numbers
    }
}
```

```

        System.out.println("First " + n + " Binary Numbers:");
        generateBinaryNumbers(n);
    }
}

```

```

package com.datastructures.Queue;

import org.junit.jupiter.api.Test;
import java.util.LinkedList;
import java.util.Queue;
import static org.junit.jupiter.api.Assertions.*;

class BinaryNumberGeneratorTest {

    @Test
    void testGenerateBinaryNumbers() {
        int n = 5;

        // Expected output for the first 5 binary numbers
        String[] expectedOutput = {"1", "10", "11", "100", "101"};

        // Creating a queue to simulate the generation process
        Queue<String> queue = new LinkedList<>();
        queue.add("1");

        // Generating binary numbers using a loop similar to the actual method
        String[] generatedOutput = new String[n];
        for (int i = 0; i < n; i++) {
            String current = queue.remove();
            generatedOutput[i] = current;
            queue.add(current + "0");
            queue.add(current + "1");
        }

        // Verifying that the generated output matches the expected output
        assertEquals(expectedOutput, generatedOutput, "The generated binary numbers should match the expected output");
    }

    @Test
    void testGenerateBinaryNumbersSingle() {
        int n = 1;

        // Expected output for the first binary number
    }
}

```

```
String[] expectedOutput = {"1"};

// Creating a queue to simulate the generation process
Queue<String> queue = new LinkedList<>();
queue.add("1");

// Generating the first binary number
String[] generatedOutput = new String[n];
generatedOutput[0] = queue.remove();

// Verifying that the generated output matches the expected output
assertArrayEquals(expectedOutput, generatedOutput, "The generated binary
number should be '1'");
}

@Test
void testGenerateBinaryNumbersZero() {
    int n = 0;

    // Expected output for zero binary numbers
    String[] expectedOutput = {};

    // Verifying that an empty output is expected for n = 0
    assertArrayEquals(expectedOutput, new String[0], "There should be no binary
numbers generated for n = 0");
}
}
```

3. Hospital Triage System

Simulate a hospital triage system using a PriorityQueue where patients with higher severity are treated first.

Example:

Patients: [("John", 3), ("Alice", 5), ("Bob", 2)] → Order: Alice, John, Bob.

```
package com.datastructures.queue;

import java.util.PriorityQueue;
import java.util.Comparator;

// Class to simulate a hospital triage system
public class HospitalTriageSystem {
```



```
// Main method to demonstrate the hospital triage system
public static void main(String[] args) {
    // Creating a PriorityQueue with a custom comparator for severity (higher
    severity first)
    PriorityQueue<Patient> triageQueue = new
    PriorityQueue<>(Comparator.comparingInt(Patient::getSeverity).reversed());

    // Adding patients to the queue
    triageQueue.add(new Patient("Kisna", 3));
    triageQueue.add(new Patient("Deepanshu", 5));
    triageQueue.add(new Patient("Raj", 2));

    // Displaying and treating patients based on severity
    System.out.println("Patients treated in order of severity:");
    while (!triageQueue.isEmpty()) {
        System.out.println(triageQueue.remove());
    }
}
}
```

```
package com.datastructures.Queue;

import com.datastructures.queue.Patient;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.PriorityQueue;
import java.util.Comparator;
import static org.junit.jupiter.api.Assertions.*;

class HospitalTriageSystemTest {
    private PriorityQueue<Patient> triageQueue;

    @BeforeEach
    void setUp() {
        // Initializing the triage queue with a custom comparator for severity
        (higher severity first)
        triageQueue = new
        PriorityQueue<>(Comparator.comparingInt(Patient::getSeverity).reversed());
    }

    @Test
    void testPatientOrderBySeverity() {
        // Adding patients to the queue
        triageQueue.add(new Patient("Kisna", 3));
```

```

    triageQueue.add(new Patient("Deepanshu", 5));
    triageQueue.add(new Patient("Raj", 2));

    // Verifying that patients are treated in the order of their severity
    assertEquals("Patient{name='Deepanshu', severity=5}",
    triageQueue.remove().toString(), "First patient should be Deepanshu with severity 5");
    assertEquals("Patient{name='Kisna', severity=3}",
    triageQueue.remove().toString(), "Second patient should be Kisna with severity 3");
    assertEquals("Patient{name='Raj', severity=2}",
    triageQueue.remove().toString(), "Third patient should be Raj with severity 2");
    }

    @Test
    void testEmptyQueue() {
        // Verifying that the queue is initially empty
        assertTrue(triageQueue.isEmpty(), "Queue should be empty initially");

        // Attempting to remove from an empty queue
        Exception exception = assertThrows(Exception.class, () -> {
            triageQueue.remove();
        });
        assertTrue(exception instanceof java.util.NoSuchElementException, "Should throw NoSuchElementException when removing from an empty queue");
    }

    @Test
    void testSinglePatient() {
        // Adding a single patient
        triageQueue.add(new Patient("Yogesh", 4));

        // Verifying the order for a single patient
        assertEquals("Patient{name='Yogesh', severity=4}",
    triageQueue.remove().toString(), "Only patient should be Yogesh with severity 4");
        assertTrue(triageQueue.isEmpty(), "Queue should be empty after removing the only patient");
    }
}

```

4. Implement a Stack Using Queues

Implement a stack data structure using two queues and support push, pop, and top operations.

Example:

Push 1, 2, 3 → Pop → Output: 3.

```
package com.datastructures.queue;

import java.util.LinkedList;
import java.util.Queue;

// Class to implement a stack using two queues
public class StackUsingQueues<T> {
    private Queue<T> queue1;
    private Queue<T> queue2;

    // Constructor to initialize two queues
    public StackUsingQueues() {
        queue1 = new LinkedList<>();
        queue2 = new LinkedList<>();
    }

    // Method to push an element onto the stack
    public void push(T element) {
        // Adding the element to queue1
        queue1.add(element);
    }

    // Method to pop an element from the stack
    public T pop() {
        if (queue1.isEmpty()) {
            throw new IllegalStateException("Stack is empty");
        }

        // Moving elements from queue1 to queue2, leaving the last element
        while (queue1.size() > 1) {
            queue2.add(queue1.remove());
        }

        // The last element of queue1 is the top of the stack
        T topElement = queue1.remove();

        // Swap the references of queue1 and queue2
        Queue<T> temp = queue1;
        queue1 = queue2;
        queue2 = temp;

        return topElement;
    }
}
```

```
// Method to get the top element without removing it
public T top() {
    if (queue1.isEmpty()) {
        throw new IllegalStateException("Stack is empty");
    }

    // Moving elements to queue2 and tracking the last element
    T topElement = null;
    while (!queue1.isEmpty()) {
        topElement = queue1.remove();
        queue2.add(topElement);
    }

    // Swap the references of queue1 and queue2
    Queue<T> temp = queue1;
    queue1 = queue2;
    queue2 = temp;

    return topElement;
}

// Method to check if the stack is empty
public boolean isEmpty() {
    return queue1.isEmpty();
}

// Main method to demonstrate stack operations
public static void main(String[] args) {
    // Creating a stack of integers
    StackUsingQueues<Integer> stack = new StackUsingQueues<>();

    // Pushing elements onto the stack
    stack.push(1);
    stack.push(2);
    stack.push(3);

    // Displaying the top element
    System.out.println("Top element: " + stack.top());

    // Popping elements from the stack
    System.out.println("Popped element: " + stack.pop());
    System.out.println("Popped element: " + stack.pop());

    // Checking if the stack is empty
```

```
        System.out.println("Is stack empty? " + stack.isEmpty());
    }
}
```

```
package com.datastructures.Queue;

import com.datastructures.queue.StackUsingQueues;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class StackUsingQueuesTest {
    private StackUsingQueues<Integer> stack;

    @BeforeEach
    void setUp() {
        // Initializing the stack before each test
        stack = new StackUsingQueues<>();
    }

    @Test
    void testPushAndPop() {
        // Pushing elements onto the stack
        stack.push(10);
        stack.push(20);
        stack.push(30);

        // Verifying that elements are popped in reverse order of insertion
        assertEquals(30, stack.pop(), "First popped element should be 30");
        assertEquals(20, stack.pop(), "Second popped element should be 20");
        assertEquals(10, stack.pop(), "Third popped element should be 10");
    }

    @Test
    void testTop() {
        // Pushing elements onto the stack
        stack.push(5);
        stack.push(15);

        // Verifying that the top element is correct without removing it
        assertEquals(15, stack.top(), "Top element should be 15");
        assertEquals(15, stack.pop(), "After checking top, first popped element should still be 15");
    }
}
```

```
@Test
void testIsEmpty() {
    // Verifying that the stack is initially empty
    assertTrue(stack.isEmpty(), "Stack should be empty initially");

    // Pushing an element and verifying it's not empty
    stack.push(42);
    assertFalse(stack.isEmpty(), "Stack should not be empty after pushing an
element");

    // Popping the element and verifying the stack is empty again
    stack.pop();
    assertTrue(stack.isEmpty(), "Stack should be empty after popping the only
element");
}

@Test
void testPopFromEmptyStack() {
    // Verifying that popping from an empty stack throws an exception
    Exception exception = assertThrows(IllegalStateException.class, () -> {
        stack.pop();
    });
    assertEquals("Stack is empty", exception.getMessage());
}

@Test
void testTopFromEmptyStack() {
    // Verifying that getting the top element from an empty stack throws an
exception
    Exception exception = assertThrows(IllegalStateException.class, () -> {
        stack.top();
    });
    assertEquals("Stack is empty", exception.getMessage());
}
}
```

5. Circular Buffer Simulation

Implement a circular buffer (fixed-size queue) using an array-based queue. When full, overwrite the oldest element.

Example:

Buffer size=3: Insert 1, 2, 3 → Insert 4 → Buffer: [2, 3, 4].

```
package com.datastructures.queue;

import java.util.Arrays;

// Class to implement a circular buffer (fixed-size queue)
public class CircularBuffer<T> {
    private T[] buffer;
    private int head = 0;
    private int tail = 0;
    private int size = 0;
    private int capacity;

    // Constructor to initialize the buffer with a fixed capacity
    @SuppressWarnings("unchecked")
    public CircularBuffer(int capacity) {
        this.capacity = capacity;
        buffer = (T[]) new Object[capacity];
    }

    // Method to add an element to the buffer
    public void add(T element) {
        if (size == capacity) {
            // Overwriting the oldest element
            head = (head + 1) % capacity;
        } else {
            size++;
        }
        buffer[tail] = element;
        tail = (tail + 1) % capacity;
    }

    // Method to get all elements in the buffer in the correct order
    public T[] getElements() {
        @SuppressWarnings("unchecked")
        T[] elements = (T[]) new Object[size];
        for (int i = 0; i < size; i++) {
            elements[i] = buffer[(head + i) % capacity]; // Ensure correct wrapping
        }
        return elements;
    }

    // Method to display the elements in the buffer
    public void displayBuffer() {
```

```

        System.out.println("Buffer: " + Arrays.toString(getElements()));
    }

    // Main method to demonstrate the circular buffer
    public static void main(String[] args) {
        // Creating a circular buffer with a capacity of 3
        CircularBuffer<Integer> buffer = new CircularBuffer<>(3);

        // Adding elements to the buffer
        buffer.add(1);
        buffer.add(2);
        buffer.add(3);
        buffer.displayBuffer();

        // Adding another element, overwriting the oldest one
        buffer.add(4);
        buffer.displayBuffer();

        // Adding more elements
        buffer.add(5);
        buffer.add(6);
        buffer.displayBuffer();
    }
}

```

```

package com.datastructures.Queue;

import com.datastructures.queue.CircularBuffer;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CircularBufferTest {
    private CircularBuffer<Integer> buffer;

    @BeforeEach
    void setUp() {
        // Initializing a circular buffer with a capacity of 3
        buffer = new CircularBuffer<>(3);
    }

    @Test
    void testAddAndGetElements() {
        buffer.add(1);
    }
}

```



```

        buffer.add(2);
        buffer.add(3);

        // Verifying that the buffer contains the correct elements in the expected
order
        assertEquals(new Integer[]{1, 2, 3}, buffer.getElements(), "Buffer
should contain [1, 2, 3]");
    }

    @Test
    void testAddWithOverwrite() {
        buffer.add(1);
        buffer.add(2);
        buffer.add(3);
        buffer.add(4); // This should overwrite the oldest element (1)

        // Verifying that the buffer contains the correct elements after overwriting
        assertEquals(new Integer[]{2, 3, 4}, buffer.getElements(), "Buffer
should contain [2, 3, 4]");
    }

    @Test
    void testAddAndGetElementsMultipleOverwrites() {
        buffer.add(1);
        buffer.add(2);
        buffer.add(3);
        buffer.add(4); // Overwrite 1
        buffer.add(5); // Overwrite 2
        buffer.add(6); // Overwrite 3

        // Verifying that the buffer contains the last 3 added elements
        assertEquals(new Integer[]{4, 5, 6}, buffer.getElements(), "Buffer
should contain [4, 5, 6]");
    }

    @Test
    void testGetElementsEmptyBuffer() {
        // Verifying that an empty buffer returns an empty array
        assertEquals(new Integer[]{}, buffer.getElements(), "Buffer should be
empty");
    }

    @Test
    void testAddSingleElement() {
        buffer.add(42);
    }

```

```
// Verifying that the buffer contains only the single element
assertArrayEquals(new Integer[]{42}, buffer.getElements(), "Buffer should
contain [42]");
}
}
```

Map Interface

1. Word Frequency Counter

Read a text file and count the frequency of each word using a HashMap.
Ignore case and punctuation.

Example:

Input: "Hello world, hello Java!" → Output: {hello=2, world=1, java=1}

```
package com.datastructures.map;

import java.util.HashMap;
import java.util.Map;

// Class to count the frequency of each word in a given text
public class WordFrequencyCounter {

    // Method to count the frequency of each word in a given text
    public static Map<String, Integer> countWordFrequency(String text) {
        Map<String, Integer> wordCountMap = new HashMap<>();

        // Removing punctuation and converting to lowercase, ensuring no empty
        strings are processed
        String[] words = text.replaceAll("[^a-zA-Z ]",
        "").toLowerCase().split("\\s+");

        for (String word : words) {
            if (!word.isEmpty()) { // Ignore empty words
                wordCountMap.put(word, wordCountMap.getOrDefault(word, 0) + 1);
            }
        }
    }
}
```

```

    }
    return wordCountMap;
}

// Main method to demonstrate counting word frequency
public static void main(String[] args) {
    // Sample text input
    String text = "Hello world, hello Java! Welcome to the world of Java
programming.";

    // Counting the frequency of each word in the text
    Map<String, Integer> wordFrequency = countWordFrequency(text);

    // Displaying the word frequency map
    System.out.println("Word Frequency: " + wordFrequency);
}
}

```

```

package com.datastructures.map;

import org.junit.jupiter.api.Test;
import java.util.Map;
import static org.junit.jupiter.api.Assertions.*;

class WordFrequencyCounterTest {

    @Test
    void testCountWordFrequency() {
        // Sample text input
        String text = "Hello world, hello Java! Welcome to the world of Java
programming.";

        // Counting word frequency
        Map<String, Integer> wordFrequency =
WordFrequencyCounter.countWordFrequency(text);

        // Verifying the word frequencies
        assertEquals(2, wordFrequency.get("hello"), "Frequency of 'hello' should be
2");
        assertEquals(2, wordFrequency.get("world"), "Frequency of 'world' should be
2");
        assertEquals(2, wordFrequency.get("java"), "Frequency of 'java' should be
2");
    }
}

```

```

        assertEquals(1, wordFrequency.get("welcome"), "Frequency of 'welcome' should be 1");
        assertEquals(1, wordFrequency.get("programming"), "Frequency of 'programming' should be 1");
    }

    @Test
    void testCountWordFrequencyEmptyText() {
        // Empty text input
        String text = "";

        // Counting word frequency for an empty string
        Map<String, Integer> wordFrequency =
        WordFrequencyCounter.countWordFrequency(text);

        // Verifying that the map is empty
        assertTrue(wordFrequency.isEmpty(), "Word frequency map should be empty for an empty text");
    }

    @Test
    void testCountWordFrequencySingleWord() {
        // Single word text input
        String text = "Java";

        // Counting word frequency for a single word
        Map<String, Integer> wordFrequency =
        WordFrequencyCounter.countWordFrequency(text);

        // Verifying the frequency of the single word
        assertEquals(1, wordFrequency.get("java"), "Frequency of 'java' should be 1");
    }
}

```

2. Invert a Map

Invert a Map<K, V> to produce a Map<V, K>. Handle duplicate values by storing them in a list.

Example:

Input: {A=1, B=2, C=1} → Output: {1=[A, C], 2=[B]}.

```

package com.datastructures.map;

import java.util.ArrayList;

```

```
import java.util.HashMap;
import java.util.List;
import java.util.Map;

// Class to invert a map from K -> V to V -> List<K>
public class MapInverter {

    // Method to invert a map
    public static <K, V> Map<V, List<K>> invertMap(Map<K, V> map) {
        Map<V, List<K>> invertedMap = new HashMap<>();

        // Iterating through the original map
        for (Map.Entry<K, V> entry : map.entrySet()) {
            V value = entry.getValue();
            K key = entry.getKey();

            // Adding the key to the list of keys for the corresponding value
            invertedMap.computeIfAbsent(value, k -> new ArrayList<>()).add(key);
        }

        return invertedMap;
    }

    // Main method to demonstrate inverting a map
    public static void main(String[] args) {
        // Creating a sample map
        Map<String, Integer> map = new HashMap<>();
        map.put("A", 1);
        map.put("B", 2);
        map.put("C", 1);
        map.put("D", 3);

        // Inverting the map
        Map<Integer, List<String>> invertedMap = invertMap(map);

        // Displaying the inverted map
        System.out.println("Inverted Map: " + invertedMap);
    }
}
```

```
package com.datastructures.map;
```

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.*;
import static org.junit.jupiter.api.Assertions.*;

class MapInverterTest {
    private Map<String, Integer> map;

    @BeforeEach
    void setUp() {
        // Initializing a sample map before each test
        map = new HashMap<>();
        map.put("A", 1);
        map.put("B", 2);
        map.put("C", 1);
        map.put("D", 3);
    }

    @Test
    void testInvertMap() {
        // Inverting the map
        Map<Integer, List<String>> invertedMap = MapInverter.invertMap(map);

        // Verifying the inverted map size
        assertEquals(3, invertedMap.size(), "Inverted map should contain 3 keys");

        // Checking the list of keys for each value
        assertEquals(List.of("A", "C"), invertedMap.get(1), "Value 1 should map to keys A and C");
        assertEquals(List.of("B"), invertedMap.get(2), "Value 2 should map to key B");
        assertEquals(List.of("D"), invertedMap.get(3), "Value 3 should map to key D");
    }

    @Test
    void testInvertMapWithEmptyMap() {
        Map<String, Integer> emptyMap = new HashMap<>();

        // Inverting an empty map
        Map<Integer, List<String>> invertedMap = MapInverter.invertMap(emptyMap);

        // Verifying that the inverted map is empty
        assertTrue(invertedMap.isEmpty(), "Inverted map should be empty for an empty input map");
    }
}
```

```

}

@Test
void testInvertMapWithSingleEntry() {
    Map<String, Integer> singleEntryMap = new HashMap<>();
    singleEntryMap.put("X", 42);

    // Inverting a single-entry map
    Map<Integer, List<String>> invertedMap =
MapInverter.invertMap(singleEntryMap);

    // Verifying the inverted map
    assertEquals(1, invertedMap.size(), "Inverted map should contain 1 key");
    assertEquals(List.of("X"), invertedMap.get(42), "Value 42 should map to key
X");
}
}

```

3. Find the Key with the Highest Value

Given a Map<String, Integer>, find the key with the maximum value.

Example:

Input: {A=10, B=20, C=15} → Output: B.

```

package com.datastructures.map;

import java.util.Collections;
import java.util.HashMap;
import java.util.Map;

// Class to find the key with the highest value in a map
public class MaxValueKeyFinder {

    // Method to find the key with the highest value
    public static <K> K findKeyWithMaxValue(Map<K, Integer> map) {
        if (map.isEmpty()) {
            throw new IllegalArgumentException("The map is empty.");
        }

        // Finding the key with the maximum value
        return Collections.max(map.entrySet(),
Map.Entry.comparingByValue()).getKey();
    }

    // Main method to demonstrate finding the key with the highest value
}

```

```
public static void main(String[] args) {
    // Creating a sample map
    Map<String, Integer> scores = new HashMap<>();
    scores.put("Deepanshu", 85);
    scores.put("Raj", 92);
    scores.put("Shubham", 88);
    scores.put("Gagan", 95);

    // Finding the key with the highest value
    String topScorer = findKeyWithMaxValue(scores);

    // Displaying the key with the highest value
    System.out.println("Key with the highest value: " + topScorer);
}
}
```

```
package com.datastructures.map;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.HashMap;
import java.util.Map;
import static org.junit.jupiter.api.Assertions.*;

class MaxValueKeyFinderTest {
    private Map<String, Integer> scores;

    @BeforeEach
    void setUp() {
        // Initializing a sample map with scores
        scores = new HashMap<>();
        scores.put("Deepanshu", 85);
        scores.put("Raj", 92);
        scores.put("Shubham", 88);
        scores.put("Gagan", 95);
    }

    @Test
    void testFindKeyWithMaxValue() {
        // Finding the key with the highest value
        String result = MaxValueKeyFinder.findKeyWithMaxValue(scores);

        // Verifying that the key with the highest value is "Gagan"
        assertEquals("Gagan", result, "Key with the highest value should be");
    }
}
```



```
'Gagan'");
}

@Test
void testFindKeyWithMaxValueSingleEntry() {
    Map<String, Integer> singleEntryMap = new HashMap<>();
    singleEntryMap.put("Yogesh", 100);

    // Finding the key in a single-entry map
    String result = MaxValueKeyFinder.findKeyWithMaxValue(singleEntryMap);

    // Verifying the result
    assertEquals("Yogesh", result, "Key with the highest value should be 'Yogesh'");
}

@Test
void testFindKeyWithMaxValueEmptyMap() {
    Map<String, Integer> emptyMap = new HashMap<>();

    // Verifying that an exception is thrown for an empty map
    Exception exception = assertThrows(IllegalArgumentException.class, () ->
        MaxValueKeyFinder.findKeyWithMaxValue(emptyMap)
    );
    assertEquals("The map is empty.", exception.getMessage());
}
}
```

4. Merge Two Maps

Merge two maps such that if a key exists in both, sum their values.

Example:

Map1: {A=1, B=2}, Map2: {B=3, C=4} → Output: {A=1, B=5, C=4}.

```
package com.datastructures.map;

import java.util.HashMap;
import java.util.Map;

// Class to merge two maps and sum values for duplicate keys
public class MapMerger {

    // Method to merge two maps and sum values for keys that exist in both
    public static Map<String, Integer> mergeMaps(Map<String, Integer> map1,
        Map<String, Integer> map2) {
```

```

    Map<String, Integer> mergedMap = new HashMap<>(map1);

    // Iterating through the second map
    for (Map.Entry<String, Integer> entry : map2.entrySet()) {
        // Summing values if the key already exists
        mergedMap.merge(entry.getKey(), entry.getValue(), Integer::sum);
    }

    return mergedMap;
}

// Main method to demonstrate merging two maps
public static void main(String[] args) {
    // Creating the first sample map
    Map<String, Integer> map1 = new HashMap<>();
    map1.put("A", 1);
    map1.put("B", 2);
    map1.put("C", 3);

    // Creating the second sample map
    Map<String, Integer> map2 = new HashMap<>();
    map2.put("B", 3);
    map2.put("C", 4);
    map2.put("D", 5);

    // Merging the two maps
    Map<String, Integer> mergedMap = mergeMaps(map1, map2);

    // Displaying the merged map
    System.out.println("Merged Map: " + mergedMap);
}
}

```

```

package com.datastructures.map;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.HashMap;
import java.util.Map;
import static org.junit.jupiter.api.Assertions.*;

class MapMergerTest {
    private Map<String, Integer> map1;
    private Map<String, Integer> map2;
}

```

```
@BeforeEach
void setUp() {
    // Initializing the first sample map
    map1 = new HashMap<>();
    map1.put("A", 1);
    map1.put("B", 2);
    map1.put("C", 3);

    // Initializing the second sample map
    map2 = new HashMap<>();
    map2.put("B", 3);
    map2.put("C", 4);
    map2.put("D", 5);
}

@Test
void testMergeMaps() {
    // Merging the two maps
    Map<String, Integer> mergedMap = MapMerger.mergeMaps(map1, map2);

    // Verifying the size of the merged map
    assertEquals(4, mergedMap.size(), "Merged map should contain 4 keys");

    // Verifying the merged values
    assertEquals(1, mergedMap.get("A"), "Key 'A' should have value 1");
    assertEquals(5, mergedMap.get("B"), "Key 'B' should have value 5");
    assertEquals(7, mergedMap.get("C"), "Key 'C' should have value 7");
    assertEquals(5, mergedMap.get("D"), "Key 'D' should have value 5");
}

@Test
void testMergeMapsWithEmptyMap() {
    Map<String, Integer> emptyMap = new HashMap<>();

    // Merging map1 with an empty map
    Map<String, Integer> mergedMap = MapMerger.mergeMaps(map1, emptyMap);

    // Verifying that the merged map is equal to map1
    assertEquals(map1, mergedMap, "Merged map should be equal to map1 when merging with an empty map");
}

@Test
void testMergeMapsWithDisjointKeys() {
```

```
Map<String, Integer> disjointMap = new HashMap<>();
disjointMap.put("X", 10);
disjointMap.put("Y", 20);

// Merging map1 with a map containing disjoint keys
Map<String, Integer> mergedMap = MapMerger.mergeMaps(map1, disjointMap);

// Verifying the merged map size
assertEquals(5, mergedMap.size(), "Merged map should contain 5 keys");
assertEquals(10, mergedMap.get("X"), "Key 'X' should have value 10");
assertEquals(20, mergedMap.get("Y"), "Key 'Y' should have value 20");
}
}
```

5. Group Objects by Property

Given a list of Employee objects, group them by their department using a Map<Department, List<Employee>>.

Example:

Employees: [Alice (HR), Bob (IT), Carol (HR)] → Output: HR: [Alice, Carol], IT: [Bob].

```
package com.datastructures.map;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import static org.junit.jupiter.api.Assertions.*;

class EmployeeGrouperTest {
    private List<Employee> employees;

    @BeforeEach
    void setUp() {
        // Initializing a list of employees before each test
        employees = new ArrayList<>();
        employees.add(new Employee("Shubh", "HR"));
        employees.add(new Employee("Deepanshu", "IT"));
        employees.add(new Employee("Raj", "HR"));
        employees.add(new Employee("Gagan", "IT"));
        employees.add(new Employee("Shubham", "Finance"));
    }
}
```

```

@Test
void testGroupByDepartment() {
    // Grouping employees by department
    Map<String, List<Employee>> groupedEmployees =
EmployeeGrouper.groupByDepartment(employees);

    // Verifying the grouping
    assertEquals(3, groupedEmployees.size(), "There should be 3 departments");

    // Checking employees in each department
    assertEquals(2, groupedEmployees.get("HR").size(), "HR should have 2
employees");
    assertEquals(2, groupedEmployees.get("IT").size(), "IT should have 2
employees");
    assertEquals(1, groupedEmployees.get("Finance").size(), "Finance should have
1 employee");

    // Verifying employee names in the "HR" department
    List<Employee> hrEmployees = groupedEmployees.get("HR");
    assertTrue(hrEmployees.toString().contains("Shubh") &&
hrEmployees.toString().contains("Raj"),
        "HR should contain Shubh and Raj");
}

@Test
void testGroupByDepartmentWithEmptyList() {
    List<Employee> emptyList = new ArrayList<>();

    // Grouping an empty list of employees
    Map<String, List<Employee>> groupedEmployees =
EmployeeGrouper.groupByDepartment(emptyList);

    // Verifying that the map is empty
    assertTrue(groupedEmployees.isEmpty(), "Grouped map should be empty for an
empty employee list");
}

@Test
void testGroupByDepartmentSingleEmployee() {
    List<Employee> singleEmployeeList = new ArrayList<>();
    singleEmployeeList.add(new Employee("Yogesh", "Marketing"));

    // Grouping a single employee
    Map<String, List<Employee>> groupedEmployees =

```

```
EmployeeGrouper.groupByDepartment(singleEmployeeList);

    // Verifying the grouping
    assertEquals(1, groupedEmployees.size(), "There should be 1 department");
    assertEquals(1, groupedEmployees.get("Marketing").size(), "Marketing should
have 1 employee");
    assertEquals("Yogesh", groupedEmployees.get("Marketing").get(0).toString(),
        "Marketing department should contain Yogesh");
}
}
```

```
package com.datastructures.map;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import static org.junit.jupiter.api.Assertions.*;

class EmployeeGrouperTest {
    private List<Employee> employees;

    @BeforeEach
    void setUp() {
        // Initializing a list of employees before each test
        employees = new ArrayList<>();
        employees.add(new Employee("Shubh", "HR"));
        employees.add(new Employee("Deepanshu", "IT"));
        employees.add(new Employee("Raj", "HR"));
        employees.add(new Employee("Gagan", "IT"));
        employees.add(new Employee("Shubham", "Finance"));
    }

    @Test
    void testGroupByDepartment() {
        // Grouping employees by department
        Map<String, List<Employee>> groupedEmployees =
EmployeeGrouper.groupByDepartment(employees);

        // Verifying the grouping
        assertEquals(3, groupedEmployees.size(), "There should be 3 departments");

        // Checking employees in each department
```

```

        assertEquals(2, groupedEmployees.get("HR").size(), "HR should have 2
employees");
        assertEquals(2, groupedEmployees.get("IT").size(), "IT should have 2
employees");
        assertEquals(1, groupedEmployees.get("Finance").size(), "Finance should have
1 employee");

        // Verifying employee names in the "HR" department
        List<Employee> hrEmployees = groupedEmployees.get("HR");
        assertTrue(hrEmployees.toString().contains("Shubh") &&
hrEmployees.toString().contains("Raj"),
                "HR should contain Shubh and Raj");
    }

    @Test
    void testGroupByDepartmentWithEmptyList() {
        List<Employee> emptyList = new ArrayList<>();

        // Grouping an empty list of employees
        Map<String, List<Employee>> groupedEmployees =
EmployeeGrouper.groupByDepartment(emptyList);

        // Verifying that the map is empty
        assertTrue(groupedEmployees.isEmpty(), "Grouped map should be empty for an
empty employee list");
    }

    @Test
    void testGroupByDepartmentSingleEmployee() {
        List<Employee> singleEmployeeList = new ArrayList<>();
        singleEmployeeList.add(new Employee("Yogesh", "Marketing"));

        // Grouping a single employee
        Map<String, List<Employee>> groupedEmployees =
EmployeeGrouper.groupByDepartment(singleEmployeeList);

        // Verifying the grouping
        assertEquals(1, groupedEmployees.size(), "There should be 1 department");
        assertEquals(1, groupedEmployees.get("Marketing").size(), "Marketing should
have 1 employee");
        assertEquals("Yogesh", groupedEmployees.get("Marketing").get(0).toString(),
                "Marketing department should contain Yogesh");
    }
}

```

Insurance Policy Management System

Build a system for managing insurance policies where you have to:

- Store and manage policies with unique identifiers.
- Retrieve and manipulate policies based on different criteria.
- Track policies by various attributes such as policyholder name and expiry date.

Requirements:

1. Store Policies in a Map:

- Use HashMap to store policies with policy numbers as keys and policy details as values.
- Use LinkedHashMap to maintain the insertion order of policies.
- Use TreeMap to store policies sorted by expiry date.

2. Retrieve and Manipulate Policies:

1) Implement methods to:

- Retrieve a policy by its number.
- List all policies expiring within the next 30 days.
- List all policies for a specific policyholder.
- Remove policies that are expired.

```
package com.insurance.management;

import java.time.LocalDate;

// Class representing an insurance policy
public class Policy {
    private String policyNumber;
    private String policyholderName;
    private LocalDate expiryDate;
    private String coverageType;
    private double premiumAmount;

    // Constructor to initialize policy details
    public Policy(String policyNumber, String policyholderName, LocalDate
expiryDate, String coverageType, double premiumAmount) {
        this.policyNumber = policyNumber;
    }
}
```



```

        this.policyholderName = policyholderName;
        this.expiryDate = expiryDate;
        this.coverageType = coverageType;
        this.premiumAmount = premiumAmount;
    }

    // Getters for policy details
    public String getPolicyNumber() {
        return policyNumber;
    }

    public String getPolicyholderName() {
        return policyholderName;
    }

    public LocalDate getExpiryDate() {
        return expiryDate;
    }

    public String getCoverageType() {
        return coverageType;
    }

    public double getPremiumAmount() {
        return premiumAmount;
    }

    @Override
    public String toString() {
        return "Policy{" +
            "policyNumber='" + policyNumber + '\'' +
            ", policyholderName='" + policyholderName + '\'' +
            ", expiryDate=" + expiryDate +
            ", coverageType='" + coverageType + '\'' +
            ", premiumAmount=" + premiumAmount +
            '}';
    }
}

```

```

package com.insurance.management;

import java.time.LocalDate;
import java.util.HashSet;

```

```
import java.util.Set;

// Class to manage insurance policies
public class PolicyManager {
    private Set<Policy> policies;

    // Constructor to initialize the policy set
    public PolicyManager() {
        policies = new HashSet<>();
    }

    // Method to add a policy
    public void addPolicy(Policy policy) {
        policies.add(policy);
    }

    // Method to retrieve all policies
    public Set<Policy> getAllPolicies() {
        return policies;
    }

    // Method to retrieve policies expiring within the next 30 days
    public Set<Policy> getPoliciesExpiringSoon() {
        Set<Policy> expiringPolicies = new HashSet<>();
        LocalDate today = LocalDate.now();
        for (Policy policy : policies) {
            if (policy.getExpiryDate().isBefore(today.plusDays(30)) &&
policy.getExpiryDate().isAfter(today)) {
                expiringPolicies.add(policy);
            }
        }
        return expiringPolicies;
    }
}
```

```
package com.insurance.management;

import java.time.LocalDate;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.TreeSet;

import com.insurance.comparison.PolicyPerformanceComparer;
```

```
public class Main {
    public static void main(String[] args) {
        PolicyManager manager = new PolicyManager();

        // Adding sample policies using preferred names
        manager.addPolicy(new Policy("P1001", "Raj Verma", LocalDate.of(2025, 3,
15), "Health", 1200.50));
        manager.addPolicy(new Policy("P1002", "Gagan Sharma", LocalDate.of(2025, 2,
28), "Auto", 800.00));
        manager.addPolicy(new Policy("P1003", "Shubham Jain", LocalDate.of(2025, 1,
20), "Home", 1500.75));
        manager.addPolicy(new Policy("P1004", "Deepanshu Malviya",
LocalDate.of(2025, 4, 10), "Health", 1000.00));
        manager.addPolicy(new Policy("P1005", "Karan Mehta", LocalDate.of(2025, 1,
25), "Auto", 900.00));

        // Displaying all policies
        System.out.println("All Policies:");
        for (Policy policy : manager.getAllPolicies()) {
            System.out.println(policy);
        }

        // Displaying policies expiring within the next 30 days
        System.out.println("\nPolicies Expiring Soon:");
        for (Policy policy : manager.getPoliciesExpiringSoon()) {
            System.out.println(policy);
        }
    }
}
```

```
package com.insurance.comparison;

import com.insurance.management.Policy;

import java.time.LocalDate;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.TreeSet;

// Class to compare the performance of HashSet, LinkedHashSet, and TreeSet
public class PolicyPerformanceComparer {
```

```
// Method to add policies to a set and measure the time taken
public static void measureAddPerformance(Set<Policy> set, int numberOfPolicies)
{
    long startTime = System.nanoTime();
    for (int i = 1; i <= numberOfPolicies; i++) {
        set.add(new Policy("P" + i, "Raj", LocalDate.of(2025, 3, 15), "Health",
1000.00 + i));
    }
    long endTime = System.nanoTime();
    System.out.println(set.getClass().getSimpleName() + " - Add Performance: " +
(endTime - startTime) + " ns");
}

// Method to search for a specific policy in a set and measure the time taken
public static void measureSearchPerformance(Set<Policy> set, String
policyNumber) {
    long startTime = System.nanoTime();
    boolean found = set.stream().anyMatch(policy ->
policy.getPolicyNumber().equals(policyNumber));
    long endTime = System.nanoTime();
    System.out.println(set.getClass().getSimpleName() + " - Search Performance:
" + (endTime - startTime) + " ns | Found: " + found);
}

// Method to remove a policy from a set and measure the time taken
public static void measureRemovePerformance(Set<Policy> set, String
policyNumber) {
    long startTime = System.nanoTime();
    set.removeIf(policy -> policy.getPolicyNumber().equals(policyNumber));
    long endTime = System.nanoTime();
    System.out.println(set.getClass().getSimpleName() + " - Remove Performance:
" + (endTime - startTime) + " ns");
}
}
```

```
package com.insurance.management;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.time.LocalDate;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;
```

```
class PolicyManagerTest {
    private PolicyManager policyManager;

    @BeforeEach
    void setUp() {
        policyManager = new PolicyManager();

        // Adding sample policies with different expiry dates
        policyManager.addPolicy(new Policy("P1001", "Raj Verma",
            LocalDate.now().plusDays(10), "Health", 1200.50));
        policyManager.addPolicy(new Policy("P1002", "Gagan Sharma",
            LocalDate.now().plusDays(20), "Auto", 800.00));
        policyManager.addPolicy(new Policy("P1003", "Shubham Jain",
            LocalDate.now().plusDays(40), "Home", 1500.75));
        policyManager.addPolicy(new Policy("P1004", "Deepanshu Malviya",
            LocalDate.now().plusDays(5), "Health", 1000.00));
        policyManager.addPolicy(new Policy("P1005", "Karan Mehta",
            LocalDate.now().minusDays(10), "Auto", 900.00)); // Already expired
    }

    @Test
    void testAddPolicy() {
        int initialSize = policyManager.getAllPolicies().size();
        policyManager.addPolicy(new Policy("P1006", "Yogesh Kumar",
            LocalDate.of(2025, 5, 5), "Health", 1500.00));
        assertEquals(initialSize + 1, policyManager.getAllPolicies().size(), "Policy
count should increase by 1");
    }

    @Test
    void testGetAllPolicies() {
        Set<Policy> allPolicies = policyManager.getAllPolicies();
        assertEquals(5, allPolicies.size(), "There should be 5 policies initially");
        assertTrue(allPolicies.stream().anyMatch(p ->
p.getPolicyNumber().equals("P1001")), "Policy P1001 should exist");
        assertTrue(allPolicies.stream().anyMatch(p ->
p.getPolicyholderName().equals("Deepanshu Malviya")), "Policyholder Deepanshu
Malviya should exist");
    }

    @Test
    void testGetPoliciesExpiringSoon() {
        Set<Policy> expiringPolicies = policyManager.getPoliciesExpiringSoon();
        assertEquals(3, expiringPolicies.size(), "There should be 3 policies
expiring within the next 30 days");
    }
}
```

```

        assertTrue(expiringPolicies.stream().anyMatch(p ->
p.getPolicyNumber().equals("P1001")), "Policy P1001 should be expiring soon");
        assertTrue(expiringPolicies.stream().anyMatch(p ->
p.getPolicyNumber().equals("P1002")), "Policy P1002 should be expiring soon");
        assertTrue(expiringPolicies.stream().anyMatch(p ->
p.getPolicyNumber().equals("P1004")), "Policy P1004 should be expiring soon");
    }

    @Test
    void testGetPoliciesExpiringSoonNoneFound() {
        PolicyManager emptyManager = new PolicyManager(); // New manager with no
policies
        Set<Policy> expiringPolicies = emptyManager.getPoliciesExpiringSoon();
        assertTrue(expiringPolicies.isEmpty(), "No policies should be expiring
within the next 30 days");
    }
}

```

```

package com.insurance.comparison;

import com.insurance.management.Policy;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.time.LocalDate;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.TreeSet;

class PolicyPerformanceComparerTest {
    private Set<Policy> hashSet;
    private Set<Policy> linkedHashSet;
    private Set<Policy> treeSet;

    @BeforeEach
    void setUp() {
        hashSet = new HashSet<>();
        linkedHashSet = new LinkedHashSet<>();
        treeSet = new TreeSet<>((p1, p2) ->
p1.getPolicyNumber().compareTo(p2.getPolicyNumber()));
    }

    @Test
    void testMeasureAddPerformance() {

```

```

        System.out.println("---- Add Performance ----");
        PolicyPerformanceComparer.measureAddPerformance(hashSet, 100);
        PolicyPerformanceComparer.measureAddPerformance(linkedHashSet, 100);
        PolicyPerformanceComparer.measureAddPerformance(treeSet, 100);
    }

    @Test
    void testMeasureSearchPerformance() {
        // Adding 100 policies to each set
        for (int i = 1; i <= 100; i++) {
            Policy policy = new Policy("P" + i, "Deepanshu", LocalDate.of(2025, 3,
15), "Health", 1000.00 + i);
            hashSet.add(policy);
            linkedHashSet.add(policy);
            treeSet.add(policy);
        }

        System.out.println("\n---- Search Performance ----");
        PolicyPerformanceComparer.measureSearchPerformance(hashSet, "P50");
        PolicyPerformanceComparer.measureSearchPerformance(linkedHashSet, "P50");
        PolicyPerformanceComparer.measureSearchPerformance(treeSet, "P50");
    }

    @Test
    void testMeasureRemovePerformance() {
        // Adding 100 policies to each set
        for (int i = 1; i <= 100; i++) {
            Policy policy = new Policy("P" + i, "Gagan", LocalDate.of(2025, 3, 15),
"Health", 1000.00 + i);
            hashSet.add(policy);
            linkedHashSet.add(policy);
            treeSet.add(policy);
        }

        System.out.println("\n---- Remove Performance ----");
        PolicyPerformanceComparer.measureRemovePerformance(hashSet, "P50");
        PolicyPerformanceComparer.measureRemovePerformance(linkedHashSet, "P50");
        PolicyPerformanceComparer.measureRemovePerformance(treeSet, "P50");
    }
}

```

Design a Voting System

Description: Design a system where:

- Votes are stored in a HashMap (Candidate -> Votes).
- TreeMap is used to display the results in sorted order.
- LinkedHashMap is used to maintain the order of votes.

```
package com.votingsystem;

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.TreeMap;

// Class to simulate a voting system
public class VotingSystem {

    private Map<String, Integer> voteCount;

    // Constructor to initialize the vote count map
    public VotingSystem() {
        voteCount = new HashMap<>();
    }

    // Method to cast a vote for a candidate
    public void castVote(String candidate) {
        voteCount.put(candidate, voteCount.getOrDefault(candidate, 0) + 1);
    }

    // Method to display results in insertion order using LinkedHashMap
    public Map<String, Integer> getResultsInInsertionOrder() {
        return new LinkedHashMap<>(voteCount);
    }

    // Method to display results in sorted order using TreeMap
    public Map<String, Integer> getResultsInSortedOrder() {
        return new TreeMap<>(voteCount);
    }
}
```

```
package com.votingsystem;

public class Main {
```



```
public static void main(String[] args) {
    VotingSystem votingSystem = new VotingSystem();

    // Casting votes using preferred names
    votingSystem.castVote("Raj");
    votingSystem.castVote("Gagan");
    votingSystem.castVote("Shubham");
    votingSystem.castVote("Raj");
    votingSystem.castVote("Deepanshu");
    votingSystem.castVote("Gagan");

    // Displaying results in insertion order
    System.out.println("Results in Insertion Order:");
    votingSystem.getResultsInInsertionOrder().forEach((candidate, votes) ->
        System.out.println(candidate + ": " + votes));

    // Displaying results in sorted order
    System.out.println("\nResults in Sorted Order:");
    votingSystem.getResultsInSortedOrder().forEach((candidate, votes) ->
        System.out.println(candidate + ": " + votes));
}
}
```

```
package com.votingsystem;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.util.Map;

import static org.junit.jupiter.api.Assertions.*;

class VotingSystemTest {
    private VotingSystem votingSystem;

    @BeforeEach
    void setUp() {
        votingSystem = new VotingSystem();

        // Casting votes for initial setup
        votingSystem.castVote("Raj");
        votingSystem.castVote("Gagan");
        votingSystem.castVote("Shubham");
        votingSystem.castVote("Raj");
        votingSystem.castVote("Deepanshu");
    }
}
```

```

        votingSystem.castVote("Gagan");
    }

    @Test
    void testCastVote() {
        votingSystem.castVote("Karan");
        Map<String, Integer> results = votingSystem.getResultsInInsertionOrder();
        assertTrue(results.containsKey("Karan"), "Vote for Karan should be
recorded");
        assertEquals(1, results.get("Karan"), "Karan should have 1 vote");
    }

    @Test
    void testGetResultsInInsertionOrder() {
        Map<String, Integer> results = votingSystem.getResultsInInsertionOrder();
        assertEquals(4, results.size(), "There should be 4 unique candidates");
        assertEquals(2, results.get("Raj"), "Raj should have 2 votes");
        assertEquals(2, results.get("Gagan"), "Gagan should have 2 votes");
    }

    @Test
    void testGetResultsInSortedOrder() {
        Map<String, Integer> sortedResults = votingSystem.getResultsInSortedOrder();
        assertEquals(4, sortedResults.size(), "There should be 4 unique
candidates");
        assertEquals("Deepanshu", sortedResults.keySet().iterator().next(), "The
first candidate in sorted order should be Deepanshu");
    }

    @Test
    void testNoVotesCast() {
        VotingSystem emptyVotingSystem = new VotingSystem();
        assertTrue(emptyVotingSystem.getResultsInInsertionOrder().isEmpty(),
"Results should be empty if no votes are cast");
        assertTrue(emptyVotingSystem.getResultsInSortedOrder().isEmpty(), "Sorted
results should be empty if no votes are cast");
    }
}

```

Implement a Shopping Cart

Description:

- Use HashMap to store product prices.
- Use LinkedHashMap to maintain the order of items added.
- Use TreeMap to display items sorted by price.

```
package com.shopping.cart;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.TreeMap;

// Class to manage a shopping cart
public class ShoppingCart {
    private Map<String, Double> cart;

    // Constructor to initialize the cart using LinkedHashMap to preserve order
    public ShoppingCart() {
        cart = new LinkedHashMap<>();
    }

    // Method to add a product to the cart
    public void addProduct(String productName, double price) {
        cart.put(productName, price);
    }

    // Method to remove a product from the cart
    public void removeProduct(String productName) {
        cart.remove(productName);
    }

    // Method to display the cart in the order items were added
    public Map<String, Double> getCartInOrder() {
        return new LinkedHashMap<>(cart);
    }

    // Method to display the cart with items sorted by price
    public Map<String, Double> getCartSortedByPrice() {
```

```
// Using TreeMap with the PriceSorter comparator
TreeMap<String, Double> sortedCart = new TreeMap<>(new PriceSorter(cart));
sortedCart.putAll(cart);
return sortedCart;
}
}
```

```
package com.shopping.cart;

import java.util.Comparator;
import java.util.Map;

public class PriceSorter implements Comparator<String> {
    private Map<String, Double> productPrices;

    // Constructor to initialize with product prices
    public PriceSorter(Map<String, Double> productPrices) {
        this.productPrices = productPrices;
    }

    // Comparing products by price, and by product name if prices are equal
    @Override
    public int compare(String product1, String product2) {
        Double price1 = productPrices.get(product1);
        Double price2 = productPrices.get(product2);

        int priceComparison = price1.compareTo(price2);
        if (priceComparison != 0) {
            return priceComparison;
        }
        // If prices are equal, compare by product name to avoid key conflicts
        return product1.compareTo(product2);
    }
}
```

```
package com.shopping.cart;

public class Main {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();

        // Adding products to the cart
        cart.addProduct("Laptop", 80000.00);
        cart.addProduct("Smartphone", 30000.00);
    }
}
```

```

        cart.addProduct("Headphones", 5000.00);
        cart.addProduct("Smartwatch", 15000.00);

        // Displaying cart in the order items were added
        System.out.println("Cart in Order of Addition:");
        cart.getCartInOrder().forEach((product, price) ->
            System.out.println(product + ": ₹" + price));

        // Displaying cart sorted by price
        System.out.println("\nCart Sorted by Price:");
        cart.getCartSortedByPrice().forEach((product, price) ->
            System.out.println(product + ": ₹" + price));
    }
}

```

```

package com.shopping.cart;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.util.Map;

import static org.junit.jupiter.api.Assertions.*;

class ShoppingCartTest {
    private ShoppingCart cart;

    @BeforeEach
    void setUp() {
        cart = new ShoppingCart();

        // Adding sample products
        cart.addProduct("Laptop", 80000.00);
        cart.addProduct("Smartphone", 30000.00);
        cart.addProduct("Headphones", 5000.00);
        cart.addProduct("Smartwatch", 15000.00);
    }

    @Test
    void testAddProduct() {
        int initialSize = cart.getCartInOrder().size();
        cart.addProduct("Tablet", 25000.00);
        assertEquals(initialSize + 1, cart.getCartInOrder().size(), "Product count should increase by 1");
    }
}

```

```

@Test
void testRemoveProduct() {
    cart.removeProduct("Smartphone");
    assertFalse(cart.getCartInOrder().containsKey("Smartphone"), "Smartphone
should be removed from the cart");
}

@Test
void testGetCartInOrder() {
    Map<String, Double> cartInOrder = cart.getCartInOrder();
    assertEquals(4, cartInOrder.size(), "Cart should contain 4 products");
    assertEquals(80000.00, cartInOrder.get("Laptop"), "Laptop price should be
₹80000.00");
}

@Test
void testGetCartSortedByPrice() {
    Map<String, Double> sortedCart = cart.getCartSortedByPrice();
    assertEquals(5000.00, sortedCart.values().iterator().next(), "First product
in sorted cart should have price ₹5000.00");
    assertTrue(sortedCart.containsKey("Headphones"), "Headphones should be
present in the sorted cart");
}

@Test
void testGetCartSortedByPriceOrder() {
    Map<String, Double> sortedCart = cart.getCartSortedByPrice();
    Double[] sortedPrices = sortedCart.values().toArray(new Double[0]);
    for (int i = 1; i < sortedPrices.length; i++) {
        assertTrue(sortedPrices[i] >= sortedPrices[i - 1], "Prices should be in
ascending order");
    }
}
}

```

```

package com.shopping.cart;

import org.junit.jupiter.api.Test;

import java.util.*;

import static org.junit.jupiter.api.Assertions.*;

class PriceSorterTest {

```

```
@Test
void testPriceSorterAscendingOrder() {
    // Creating a sample map of products with their prices
    Map<String, Double> productPrices = new HashMap<>();
    productPrices.put("Laptop", 80000.00);
    productPrices.put("Smartphone", 30000.00);
    productPrices.put("Headphones", 5000.00);
    productPrices.put("Smartwatch", 15000.00);

    // Creating a TreeMap with the PriceSorter comparator to sort products by
    price
    TreeMap<String, Double> sortedCart = new TreeMap<>(new
    PriceSorter(productPrices));
    sortedCart.putAll(productPrices);

    // Extracting prices to verify the order
    List<Double> sortedPrices = new ArrayList<>(sortedCart.values());

    // Ensuring the prices are in ascending order
    for (int i = 1; i < sortedPrices.size(); i++) {
        assertTrue(sortedPrices.get(i) >= sortedPrices.get(i - 1), "Prices
    should be in ascending order");
    }
}

@Test
void testPriceSorterEqualPrices() {
    // Creating a sample map with equal prices
    Map<String, Double> productPrices = new HashMap<>();
    productPrices.put("Laptop", 50000.00);
    productPrices.put("Smartphone", 50000.00);
    productPrices.put("Tablet", 30000.00);

    // Creating a TreeMap with the PriceSorter comparator
    TreeMap<String, Double> sortedCart = new TreeMap<>(new
    PriceSorter(productPrices));
    sortedCart.putAll(productPrices);

    // Ensuring all products are present and sorted correctly
    assertEquals(3, sortedCart.size(), "The sorted cart should contain all
    products");
    assertEquals(30000.00, sortedCart.values().iterator().next(), "The first
    product should have the lowest price");
}
```

```
@Test
void testPriceSorterEmptyMap() {
    // Creating an empty map
    Map<String, Double> productPrices = new HashMap<>();

    // Creating a TreeMap with the PriceSorter comparator
    TreeMap<String, Double> sortedCart = new TreeMap<>(new
PriceSorter(productPrices));
    sortedCart.putAll(productPrices);

    // Ensuring the sorted cart is empty
    assertTrue(sortedCart.isEmpty(), "The sorted cart should be empty");
}
}
```

Implement a Banking System

Description:

- HashMap stores customer accounts (AccountNumber -> Balance).
- TreeMap sorts customers by balance.
- Queue processes withdrawal requests.

```
package com.banking.system;

import java.util.HashMap;
import java.util.Map;

// Class to manage customer accounts and balances
public class BankingSystem {
    private Map<String, Double> accounts;

    // Constructor to initialize the accounts map
    public BankingSystem() {
        accounts = new HashMap<>();
    }

    // Method to add a customer account with an initial balance
    public void addAccount(String accountNumber, double initialBalance) {
```



```

        accounts.put(accountNumber, initialBalance);
    }

    // Method to get the map of accounts
    public Map<String, Double> getAccounts() {
        return accounts;
    }
}

```

```

package com.banking.system;

import java.util.Map;
import java.util.Queue;

// Class to process withdrawal requests
public class WithdrawalProcessor {

    // Method to process withdrawal requests
    public static void processWithdrawals(Queue<String> withdrawalRequests,
        Map<String, Double> accounts) {
        while (!withdrawalRequests.isEmpty()) {
            String accountNumber = withdrawalRequests.poll();

            // Checking if the account exists
            if (accounts.containsKey(accountNumber)) {
                System.out.println("Processing withdrawal for account: " +
accountNumber);
            } else {
                System.out.println("Account not found: " + accountNumber);
            }
        }
    }
}

```

```

package com.banking.system;

import java.util.LinkedList;
import java.util.Queue;

public class Main {
    public static void main(String[] args) {
        BankingSystem bankingSystem = new BankingSystem();
    }
}

```

```
// Adding customer accounts with initial balances
bankingSystem.addAccount("ACC1001", 50000.00);
bankingSystem.addAccount("ACC1002", 75000.00);
bankingSystem.addAccount("ACC1003", 30000.00);
bankingSystem.addAccount("ACC1004", 100000.00);

// Displaying accounts and balances
System.out.println("Customer Accounts:");
bankingSystem.getAccounts().forEach((account, balance) ->
    System.out.println(account + ": ₹" + balance));

// Creating a queue for withdrawal requests
Queue<String> withdrawalRequests = new LinkedList<>();
withdrawalRequests.add("ACC1001");
withdrawalRequests.add("ACC1003");
withdrawalRequests.add("ACC1005"); // This account does not exist

// Processing withdrawal requests
System.out.println("\nProcessing Withdrawal Requests:");
WithdrawalProcessor.processWithdrawals(withdrawalRequests,
bankingSystem.getAccounts());
}
}
```

```
package com.banking.system;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.Map;

import static org.junit.jupiter.api.Assertions.*;

class BankingSystemTest {
    private BankingSystem bankingSystem;

    @BeforeEach
    void setUp() {
        bankingSystem = new BankingSystem();
    }

    @Test
    void testAddAccount() {
        bankingSystem.addAccount("ACC1001", 50000.00);
    }
}
```

```

        bankingSystem.addAccount("ACC1002", 75000.00);

        Map<String, Double> accounts = bankingSystem.getAccounts();

        assertEquals(2, accounts.size(), "There should be 2 accounts");
        assertEquals(50000.00, accounts.get("ACC1001"), "Balance for ACC1001 should be ₹50000.00");
        assertEquals(75000.00, accounts.get("ACC1002"), "Balance for ACC1002 should be ₹75000.00");
    }

    @Test
    void testAddAccountOverwrite() {
        bankingSystem.addAccount("ACC1001", 50000.00);
        bankingSystem.addAccount("ACC1001", 60000.00); // Overwriting the balance

        Map<String, Double> accounts = bankingSystem.getAccounts();

        assertEquals(1, accounts.size(), "There should be 1 account");
        assertEquals(60000.00, accounts.get("ACC1001"), "Balance for ACC1001 should be ₹60000.00");
    }

    @Test
    void testGetAccountsEmpty() {
        Map<String, Double> accounts = bankingSystem.getAccounts();
        assertTrue(accounts.isEmpty(), "Accounts map should be empty initially");
    }
}

```

```

package com.banking.system;

import org.junit.jupiter.api.Test;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.Map;
import java.util.Queue;
import static org.junit.jupiter.api.Assertions.*;

class WithdrawalProcessorTest {

    @Test
    void testProcessWithdrawals() {

```

```

Map<String, Double> accounts = new HashMap<>();
accounts.put("ACC1001", 50000.00);
accounts.put("ACC1002", 75000.00);

Queue<String> withdrawalRequests = new LinkedList<>();
withdrawalRequests.add("ACC1001");
withdrawalRequests.add("ACC1002");
withdrawalRequests.add("ACC9999"); // Non-existing account

// Processing withdrawals (we'll just check for the account presence here)
WithdrawalProcessor.processWithdrawals(withdrawalRequests, accounts);

assertEquals(0, withdrawalRequests.size(), "All withdrawal requests should
be processed");
}

@Test
void testProcessWithdrawalsEmptyQueue() {
    Map<String, Double> accounts = new HashMap<>();
    Queue<String> withdrawalRequests = new LinkedList<>();

    // Processing withdrawals with an empty queue
    WithdrawalProcessor.processWithdrawals(withdrawalRequests, accounts);

    assertTrue(withdrawalRequests.isEmpty(), "Queue should remain empty");
}

@Test
void testProcessWithdrawalsNonExistingAccount() {
    Map<String, Double> accounts = new HashMap<>();
    accounts.put("ACC1001", 50000.00);

    Queue<String> withdrawalRequests = new LinkedList<>();
    withdrawalRequests.add("ACC9999"); // Non-existing account

    // Capturing the console output for verification
    WithdrawalProcessor.processWithdrawals(withdrawalRequests, accounts);

    // Verifying the queue is processed
    assertTrue(withdrawalRequests.isEmpty(), "Queue should be processed even for
non-existing accounts");
}
}

```